

UNICORN: Reasoning about Configurable System Performance through the lens of Causality

Md Shahriar Iqbal
University of South Carolina
miqbal@email.sc.edu

Rahul Krishna
IBM Research
rkrnsn@ibm.com

Mohammad Ali Javidian
Purdue University
mjavidia@purdue.edu

Baishakhi Ray
Columbia University
rayb@cs.columbia.edu

Pooyan Jamshidi
University of South Carolina
pjamshid@cse.sc.edu

Abstract

Modern computer systems are highly configurable, with the total variability space sometimes larger than the number of atoms in the universe. Understanding and reasoning about the performance behavior of highly configurable systems, due to a vast variability space, is challenging. State-of-the-art methods for performance modeling and analyses rely on predictive machine learning models, therefore, they become (i) *unreliable in unseen environments* (e.g., different hardware, workloads), and (ii) *produce incorrect explanations*. To this end, we propose a new method, called UNICORN, which (i) *captures intricate interactions* between configuration options across the software-hardware stack and (ii) describes how such interactions impact *performance variations* via causal inference. We evaluated UNICORN on six highly configurable systems, including three on-device machine learning systems, a video encoder, a database management system, and a data analytics pipeline. The experimental results indicate that UNICORN outperforms state-of-the-art performance optimization and debugging methods. Furthermore, unlike the existing methods, the learned causal performance models reliably predict performance for new environments.

1 Introduction

Modern computer systems, such as data analytics pipelines, are typically composed of multiple components, each component has many configuration options, and they can be individually deployed or collocated on hardware platforms. The configuration space of highly configurable systems is combinatorially large, with 100s if not 1000s of software and hardware configuration options that interact non-trivially with one another [40, 53, 104, 105]. Developers of individual components typically have a very local, and thus limited, understanding regarding the performance behavior of systems composing the components. Developers and users of the final system are often overwhelmed with the complexity of composing and configuring components, and thus, configuring these systems to achieve specific performance goals is challenging and error-prone.

Misconfigurations are typically caused by interactions between software and hardware, resulting in *non-functional*

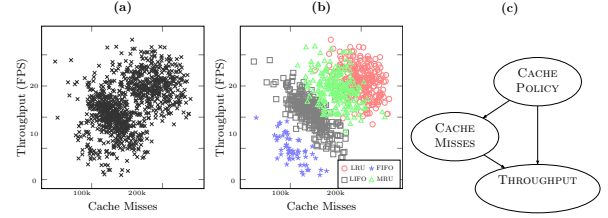


Figure 1. Simple example showing the effectiveness of causality in reasoning about system performance behavior. (a) Observational data in shows that the increase in Cache Misses leads to high Throughput and such trend is typically captured by statistical reasoning in ML models; (b) incorporating Cache Policy as a confounder correctly shows increase of Cache Misses correspond to decrease in Throughput; (c) the corresponding causal model capture Cache Policy as a common cause and explains performance behavior correctly.

*faults*¹—depredations in *non-functional* system properties such as latency and energy consumption. These non-functional faults—unlike regular software bugs—do not cause the system to crash or exhibit any obvious misbehavior [76, 85, 99]. Instead, misconfigured systems remain operational but degrade in performance [16, 71, 75, 86]. Misconfigurations caused major issues in cloud infrastructure [19], internet-scale systems [14], and on-device machine learning systems [3]. For example, a developer complained that “*I have a complicated system composed of multiple components running on NVIDIA Nano and using several sensors and I observed several performance issues. [3].*” After struggling in fixing the issues over several days, the developer concludes *there is a lot of knowledge required to optimize network stack and measure CPU load correctly. I tried to play with every configuration option explained in the kernel documents.*” In addition, they would like to *understand* the impact of configuration options and their interactions: “*What is the effect of swap memory on increasing throughput? [1]*”.

Existing works and gap. Understanding the performance behavior of configurable systems can enable (i) performance

¹we use *non-functional* and *performance faults* interchangeably for severe performance degradation that are caused by certain type of misconfigurations, aka specious configuration [49].

debugging [36, 92], (ii) performance tuning [44, 47, 48, 73, 74, 79, 97, 101, 107], and (iii) architecture adaptation [7, 27, 28, 32, 46, 55, 58, 61]. A common strategy to build performance influence models—regression models that explain the influence of individual options and their interactions [38, 83, 92, 101]. These approaches are adept at inferring the correlations between configuration options and performance objectives, however, performance influence models suffer from several shortcomings (see Section 2): (i) they become *unreliable in unseen environments* and (ii) *produce incorrect explanations*.

Our approach. Based on the intuition and several experimental evidence presented in the following section, this paper proposes UNICORN—a methodology that enables reasoning about configurable system performance with causal inference and counterfactual reasoning. UNICORN first recovers the underlying causal structure from performance data. The causal performance model allows users to (a) identify the root causes of performance faults, (b) estimate the causal effects of various configurable parameters on the performance objectives, and (c) prescribe candidate configurations to fix the performance fault or optimize system performance.

Contributions. Our contributions are as follows.

- We propose UNICORN (Section 3), a novel approach that allows causal reasoning about system performance.
- We rigorously evaluate UNICORN in a controlled case study (Section 4) as well as real-world large-scale experiments. In particular, we evaluated *effectiveness* (Section 6), *transferability* (Section 7), and *scalability* (Section 8) by comparing UNICORN with state-of-the-art performance debugging (CBI [95], DD [8], ENCORE [110], and BUGDOC [68]) and optimization (SMAC [50] and PESMO [45]). We performed the evaluation using six real-world highly configurable systems, including a video analytic pipeline, Deepstream [4], three deep learning-based systems, XCEPTION [18], DEEPSPEECH [43], and BERT [25], a video encoder, X264 [6], and a database engine, SQLite [5], deployed on NVIDIA JETSON hardware (TX1, TX2, and XAVIER).
- In addition to sample efficiency and accuracy of UNICORN in finding root causes of performance issues, we show that the learned causal performance model is transferable across different workload and deployment environments. Finally, we demonstrate the scalability of UNICORN scales to large systems consisting of 500 options and several trillion potential configurations.
- Code and the experimental data are available at <https://github.com/softsys4ai/unicorn>

2 Causal Performance Modeling and Analyses: Motivating Scenarios

In this section, we, first, point out reliability and stability issues with existing performance analyses practices via a simple as well as a large-scale real scenario. We then define a new abstraction that enables us to perform causal reasoning

in systems. We, finally, show the effectiveness of causality vs correlation for performance tasks that address the issues.

A simple scenario: Fig. 1 shows a simple scenario where the observational data indicates that Throughput is positively correlated with increased Cache Misses (as in Fig. 1 (a)). A simple ML model built on this data will predict with high confidence that larger Cache Misses leads to higher Throughput—this is misleading as higher Cache Misses should, in theory, lower Throughput. However, segregating the same data on Cache Policy (as in Fig. 1 (b)) reveals that within each group of Cache Misses, as Cache Misses increases, the Throughput decreases. One would expect such behavior as the more Cache Misses the higher number of access to external memory and therefore, the Throughput would be expected to decrease. The system resource manager may change the Cache Policy based on some criteria; this means that for the same number of Cache Misses, the Throughput may be lower or higher, however, in all Cache Policies, the increases of Cache Misses result in a decrease in Throughput. Thus, Cache Policy acts as a confounder that explains the relation between Cache Misses and Throughput, which a correlation-based model will not be able to capture. In contrast, a causal performance model, as shown in Fig. 1 (c), finds the relation between Cache Misses, Cache Policy, and Throughput and thus can reason about the observed behavior correctly.

In reality, performance analysis and debugging of heterogeneous multi-component systems is non-trivial, and often compared with finding the needle in the haystack [106]. In particular, there are two main challenges: (i) end-to-end performance analysis is not possible by reasoning about individual components in isolation, (ii) intra-component analysis may not be sufficient as severe performance bottlenecks may happen due to suboptimal interactions of configuration options across components. Next, we use a highly configurable multi-stack system to motivate why causal reasoning is a better choice for understanding the performance behavior of complex systems and then illustrate UNICORN with the example.

Scenario involving highly configurable multi-stack system:

We deployed a data analytics pipeline, DEEPSTREAM [4], on NVIDIA JETSON XAVIER hardware as shown in Fig. 2. DEEPSTREAM has many components, and each component has many configuration options, resulting in many variants of the same system. In particular, the variability comes from: (i) the configuration options of each software component in the pipeline, (ii) configurable low-level libraries that implement functionalities required by different components (e.g., the choice of tracking algorithm in the tracker or different neural architectures), (iii) the configuration options associated with each component's deployment stack (e.g., CPU Frequency of XAVIER). Further, there exist many configurable events that can be measured/observed at the OS

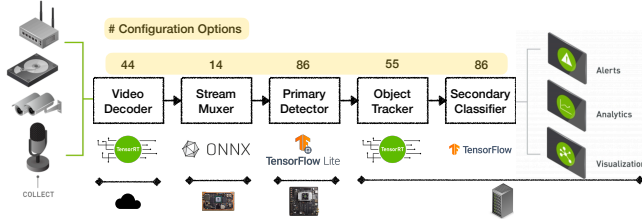


Figure 2. DEEPSTREAM: An example of a highly-configurable composed system—A big data analytics pipeline system, with several configurable components: a (i) Video Decoder, which performs video encoding/decoding with different formats; (ii) Stream Muxer, a plugin that accepts input streams and converts them to sequential batch frames; (iii) Primary Detector, which transforms the input frames based on input NN requirements and does model inference to detect objects; (iv) Object Tracker, which supports multi-object tracking; (v) Secondary Classifier, which improves performance by avoiding re-inferencing on the same objects in every frame.

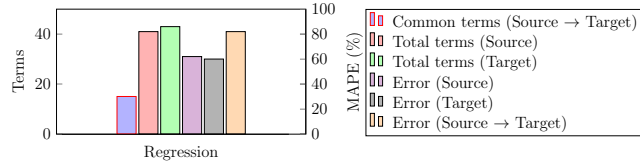


Figure 3. Regression models do not generalize well as the number of common terms, total terms and prediction error of the structural causal models change from source (XAVIER) to target (TX2). The rank correlation between source and target is 0.07 (p-value=0.73).

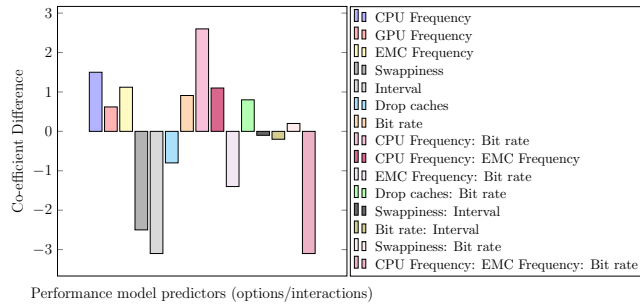


Figure 4. Visualizing co-efficient differences from the source (XAVIER) regression model to the target (TX2) regression model for the common terms and interactions (shown by ":").

level by the event tracing system. Such huge variabilities make performance analysis challenging. Moreover, configuration options among the components *interact* with each other, making the performance analysis tasks even harder.

In particular, we focus on two performance tasks: (i) *Performance Debugging*: It starts with an observed performance issue (e.g., slow execution), and the task is involved replacing the current configurations in the deployed environment with another configuration that fixes the observed performance issue; (ii) *Performance Optimization*: Here, no performance

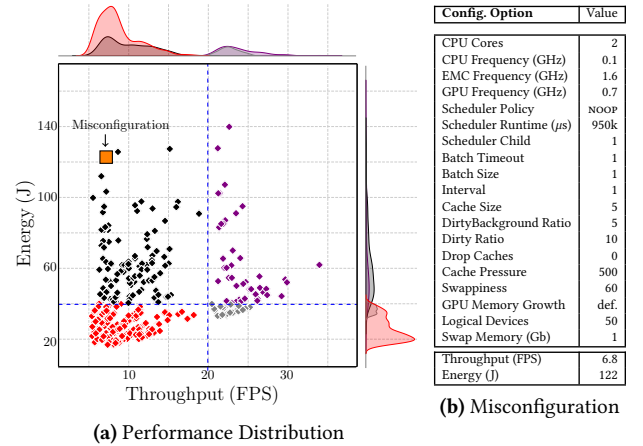


Figure 5. (a) Performance distribution when DeepStream is deployed on NVIDIA Jetson Xavier (b) Misconfiguration that caused the multi-objective non-functional fault, shown as $\square$ in the performance distribution.

issue is observed; however, one wants to get a near-optimal performance by finding a configuration that enables the best tradeoff in the multi-objective space (e.g., throughput vs. energy consumption vs. accuracy in DEEPSTREAM). To better understand the potential of the proposed approach, we show the limitations of existing correlation-based approaches for these tasks in the context of DEEPSTREAM Framework.

In particular, we measured (i) application performance metrics including throughput and energy consumption by instrumenting the DEEPSTREAM code, and (ii) 288 system-wide performance events (hardware, software, cache, and tracepoint) using *perf* and measured the data for 2000 configurations of DEEPSTREAM. We measured the performance data in two different hardware environments, JETSON XAVIER and TX2. More specifically, the configuration space of the system included (i) Software level (Decoder: 44, Stream Muxer: 14, Detector: 52), (ii) 206 Kernel level (e.g., Swappiness, Scheduler Policy, etc.), and (iii) 4 Hardware options (CPU Frequency, Active Cores, etc.). We used 8 camera streams as the workload. We used x286 as the decoder, TrafficCamNet model that uses ResNet 18 architecture for the detector. This model is pre-trained in 4 classes on a dataset of 150k frames and has an accuracy of 83.5% for detecting and tracking cars from a traffic camera’s viewpoint. The 4 classes are Vehicle, BiCycle, Person, and Roadsign. We use the Keras (TensorFlow backend) pre-trained model from TensorRT. We used NvDCF tracker, which uses a correlation filter-based online discriminative learning algorithm to a single object and uses a data association algorithm for multi-object tracking. As it is depicted in Fig. 5a, performance behavior of DEEPSTREAM, like other highly configurable systems, is non-linear, multi-modal, non-convex [54].

To show major shortcomings of existing state-of-the-art performance models, we built performance influence models

that have extensively been used in the systems' literature [35, 36, 38, 56, 60, 65, 72, 93, 94] and it is the standard approach in industry [60, 65]. Specifically, we built non-linear regression models with forward and backward elimination using a step-wise training method on the DEEPSTREAM performance data. We then performed several sensitivity analyses and identified the following issues:

1. Performance influence models could not reliably predict performance in unseen environments. Performance behavior of configurable systems varies across environments, e.g., when we deploy a software on a new hardware with a different microarchitecture or when the workload changes [51, 56–58, 101]. When building a performance model, it is important to capture predictors that transfer well, i.e., remain *stable* across environmental changes. The predictors in performance models are options (o_i) and interactions ($\phi(o_i..o_j)$) that appear in the explainable models of form $f(c) = \beta_0 + \sum_i \phi(o_i) + \sum_{i..j} \phi(o_i..o_j)$. The transferability of performance predictors is expected from performance models since they are learned based on one environment (e.g., staging as the source environment) and are desirable to reliably predict performance in another environment (e.g., production as the target environment). Therefore, if the predictors in a performance model become unstable, even if they produce accurate predictions in the current environment, there is no guarantee that it performs well in other environments, i.e., they become unreliable for performance predictions and performance optimizations due to large prediction errors. To investigate how transferable performance influence models are across environments, we performed a thorough analysis when learning a performance model for DEEPSTREAM deployed on two different hardware platforms that have two different microarchitectures. Note that such environmental changes are common, and it is known that performance behavior changes when in addition to a change of hardware resources (e.g., higher CPU Frequency), we have major differences in terms of architectural constructs [23, 26], also supported by a thorough empirical study [56]. The results in Fig. 3 indicate that the number of stable predictors is too small with respect to the total number of predictors that appear in the learned regression models.

2. Performance influence models could produce incorrect explanations. In addition to performance predictions, where developers are interested to know the effect of configuration changes on performance objectives, they are also interested to estimate and explain the effect of a change in particular *configuration options* (e.g., changing Cache Policy) toward performance variations. It is therefore desirable that the strength of the predictors in performance models, determined by their coefficients, remain consistent across environments [26, 56]. In the context of our simple scenario in Fig. 1, the performance influence model that has been learned indicates that $0.16 \times \text{Cache Misses}$ is the most influential term that determines throughput, however, the (causal) model

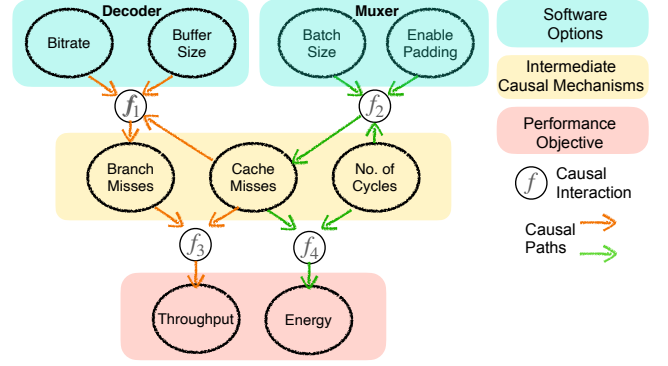


Figure 6. A partial causal performance model for DeepStream.

in Fig. 1 (c) show that there the interactions between configuration options Cache Policy and system event Cache Misses is a more reliable predictor of the throughput, indicating that the performance influence model, due to relying on superficial correlational statistics, incorrectly explains factors that influence performance behavior of the system. We performed a thorough analysis and found out that a very low Spearman rank correlation between predictors coefficients, indicating that a performance model based on regression could be highly unstable and thus would produce unreliable explanations as well as unreliable estimation of the effect of change in specific options for performance debugging purposes.

Causal Performance Models. We hypothesize that the reason behind the above-mentioned issues is the inability of correlation-based models to capture causally relevant predictors in the learned performance models. Hence, we introduce a new abstraction for performance modeling, called **Causal Performance Model (CPM)**, that gives us the leverage for performing causal reasoning in the systems' performance domain. A CPM is an instantiation of Graphical Models [80] with new types and structural constraints to enable performance modeling. Formally, CPMs (cf., Fig. 6) are Directed Acyclic Graphs (DAGs) [80] with (i) performance variables, (ii) functional nodes that define functional dependencies between performance variables, and (iii) causal links that interconnect performance nodes with each other via functional nodes, and (iv) structural constraints to define assumptions we require in performance modeling. In particular, we defined three new variable types: (1) Software-level configuration options associated with a software component in the composed system (e.g., Bitrate in the decoder component of DEEPSTREAM), (2) intermediate performance variables relating the effect of configuration options to performance objectives including middleware traces (e.g., Context Switches), performance events (e.g., Cache Misses), and hardware-level options (e.g., CPU Frequency), and (3) end-to-end performance objectives (e.g., Throughput, Energy consumption). We also characterize the functional nodes

with polynomial models to be explainable, although, in a generic form, they could be characterized with any functional forms, e.g., neural networks [87, 108]. We also defined two specific constraints over CPMs to characterize the assumptions in performance modeling: (i) defining variables that can be intervened (note that some performance variables can only be observed (e.g., Cache Misses) or in some cases where a variable can be intervened, the user may want to restrict the variability space, e.g., the cases where the user may want to use prior experience, restricting the variables that do not have a major impact to performance objectives); (ii) structural constraints, e.g., configuration options do not cause other options. Note that such constraints enable incorporating domain knowledge and enable further sparsity that facilitates learning with low sample sizes.

How causal reasoning can fix the reliability and explainability issues in current performance analyses practices. The causal performance models contain more detail than the joint distribution of all variables in the model. For example, the CPM in Fig. 6 encodes not only Branch Misses and Throughput readings are dependent but also that lowering Cache Misses causes the Throughput of DEEPSTREAM to increase and not the other way around. The arrows in causal performance models correspond to the assumed direction of causation, and the absence of an arrow represents the absence of direct causal influence between variables, including configuration options, system events, and performance objectives. The only way we can make predictions about how performance distribution changes for a system when deployed in another environment or when its workload changes are if we know how the variables are causally related. This information about causal relationships is not captured in non-causal models, such as regression-based models. Using the encoded information in CPMs, we can benefit from analyses that are only possible when we explicitly employ causal models, in particular, interventional and counterfactual analyses [81, 82]. For example, imagine that in a hardware platform, we deploy the DEEPSTREAM and observed that the system throughput is below 30/s and Buffer Size as one of the configuration options was determined dynamically between 8k-20k. The system maintained may be interested in estimating the likelihood of fixing the performance issue in a counterfactual world where the Buffer Size is set to a fixed value, 6k. The estimation of this counterfactual query is only possible if we have access to the underlying causal model because setting a specific option to a fixed value is an intervention as opposed to conditional observations that have been done in the traditional performance model for performance predictions.

Causal performance models are not only capable of predicting system performance in certain environments, they encode the causal structure of the underlying system performance behavior, i.e., the data-generating mechanism behind

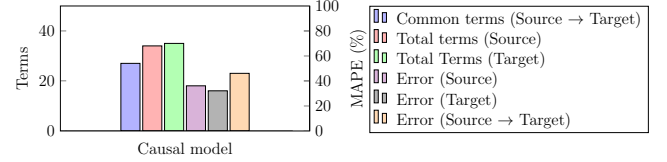


Figure 7. Causal models generalize better as the number of common terms, total terms and prediction error of the structural does not change much from source (XAVIER) to target (TX2). The rank correlation between source and target is 0.49 (p-value=0.76).

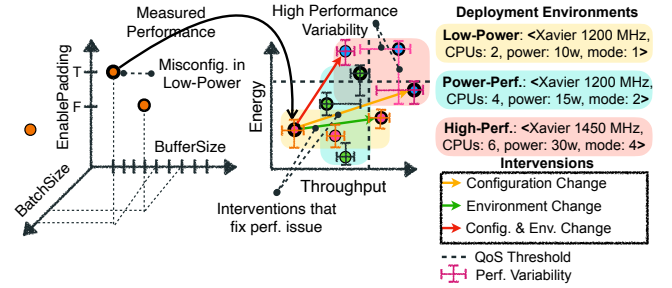


Figure 8. Mapping configuration space to multi-objective performance space.

system performance. Therefore, the causal model can reliably transfer across environments [88]. To demonstrate this for CPMs as a particular characterization of causal models, we performed a similar sensitivity analysis to regression-based models and observed that CPMs can reliably predict performance in unseen environments (see Fig. 7). In addition, as opposed to PIMs that are only capable of performance predictions, CPMs can be used for several downstream heterogeneous performance tasks. For example, using a CPM, we can determine the *causal effects* of configuration options on performance objectives. Using the estimated causal effects, one can determine the effect of change in a particular set of options towards performance objectives and therefore can select the options with the highest effects to fix a performance issue, i.e., bring back the performance objective that has violated a specific quality of service constraint without sacrificing other objectives. CPMs are also capable of predicting performance behavior by calculating conditional expectation, $E(Y|X)$, where Y indicates performance objectives, e.g., throughput, and $X = x$ is the system configurations that have not been measured.

3 UNICORN

This section presents UNICORN—our methodology for performance analyses of highly configurable and composable systems with causal reasoning.

Overview. UNICORN works in five stages, implementing an active learning loop (cf. Fig. 9): (i) Users or developers of a highly-configurable system *specify*, in a human-readable

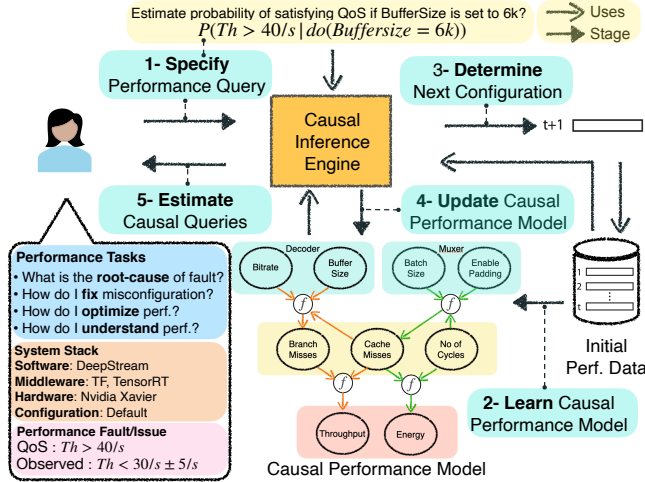


Figure 9. Overview of UNICORN.

language, the performance task at hand in terms of a query in the Inference Engine. For example, a DeepStream user may have experienced a throughput drop when they have deployed it on NVIDIA Xavier in low-power mode (cf. Fig. 8). Then, UNICORN’s main process starts by (ii) collecting some predetermined number of samples and *learning a causal performance model*; Here, a sample contains a system configuration and its corresponding measurement—including low-level system events and end-to-end system performance. Given a certain budget, which in practice either translates to time [52] or several samples [54], UNICORN, at each iteration, (iii) *determines the next configuration(s)* and measures system performance when deployed with the determined configuration—i.e. new sample; accordingly, (iv) the *learned causal performance model is incrementally updated*, reflecting a model that captures the underlying causal structure of the system performance. UNICORN terminates if either budget is exhausted or the same configuration has been selected a certain number of times consecutively, otherwise, it continues from stage-iii. Finally, (v) to automatically derive the quantities which are needed to conduct the performance tasks, the specified performance queries are *translated* to formal causal queries, and they will be *estimated* based on the final causal model.

Stage-I: Formulate Performance Queries. UNICORN enables *developers* and *users* of highly-configurable systems to conduct performance tasks, including performance debugging, optimization, and tuning in the following scenarios. In particular, they need to answer several performance queries: (i) What configuration options *caused* the performance fault? (ii) What are *important options and their interactions* that influence performance? (iii) How to *optimize* one quality or navigate *tradeoffs* among multiple qualities in a reliable and

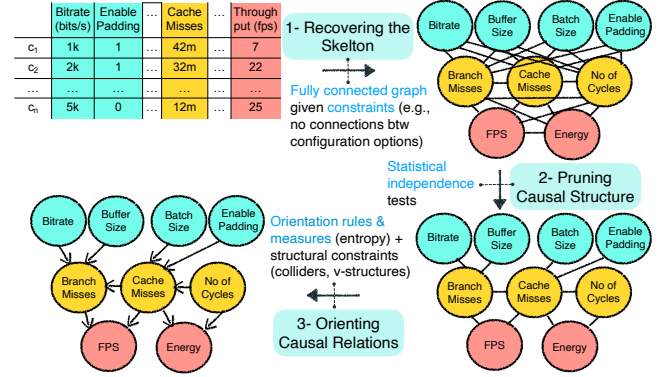


Figure 10. Causal model learning from performance data.

explainable fashion? (iv) How can we *understand* what options and possible interactions are most responsible for the performance degradation in production?

At this stage, the performance queries are translated to formal causal queries using the interface of the causal inference engine in UNICORN. Note that in the current implementation of UNICORN, this translation is performed manually, however, this process could be made automatically by creating a grammar for specifying performance queries and the translations can be made between the performance query into the well-defined causal queries, note that such translation has been done in domains such as genomics [29].

Stage-II: Learn Causal Performance Model In this stage, UNICORN learns a CPM (see Section 2) that explains the causal relations between configuration options, the intermediate causal mechanism, and performance objectives. Here, we use an existing structure learning algorithm called *Fast Causal Inference* (hereafter, FCI) [96]. We selected FCI because: (i) it accommodates for the existence of unobserved confounders [34, 78, 96], i.e., it operates even when there are latent common causes that have not been, or cannot be, measured. This is important because we do not assume absolute knowledge about configuration space, hence there could be certain configurations we could not modify or system events we have not observed. (ii) FCI, also, accommodates variables that belong to various data types such as nominal, ordinal, and categorical data common across the system stack (cf. Fig. 8). To build the CPM, we, first, gather a set of initial samples (cf. Table Fig. 10). To ensure reliability [23, 26], we measure each configuration multiple times, and we use the median for the causal model learning. As depicted in Fig. 10, UNICORN implements three steps for causal structure learning: (i) recovering the skeleton of the CPM by enforcing structural constraints; (ii) pruning the recovered structure using standard statistical tests of independence. In particular, we use mutual info for discrete variables and Fisher z-test for continuous variables. (iii) orienting undirected edges using entropy [20, 21, 34, 78, 96].

Stage-III: Iterative Sampling At this stage, UNICORN determines the next configuration to be measured. UNICORN first estimates the causal effects of configuration options towards performance objectives using the learned causal performance model. Then, UNICORN iteratively determines the next system configuration using the estimated causal effects as a heuristic. Specifically, UNICORN determines the value assignments for options with a probability that is determined proportionally based on their associated causal effects. The key intuition is that such changes in the options are more likely to have a larger effect on performance objectives, and therefore we can learn more about the performance behavior of the system. Given the exponentially large configuration space and the fact that the span of performance variations is determined by a small percentage of configurations, if we had ignored such estimates for determining the change in configuration options, the next configurations would result in considerable variations in performance objectives comparing with the existing data. Therefore, measuring the next configuration would not provide additional information for the causal model.

Stage-IV: Update Causal Performance Model At each iteration, UNICORN measures the configuration that is determined by Phase-III and updates the causal performance model (CPM) incrementally. Since the causal model uses limited observational data, there may be a discrepancy between the underlying performance model and the learned CPM, note that this issue exist in all domains using data-driven models, including causal reasoning [81]. The more accurate the causal graph, the more accurate the proposed intervention will be [20, 21, 34, 78, 96]. Figure 12 shows an example of an iterative decrease of structural hamming distance (SHD) [77] between the learned causal model and (approximate) ground truth causal model. Therefore, in case our repairs do not fix the faults, we update the observational data with this new configuration and repeat the process. Over time, the estimations of causal effects will become more accurate. We terminate the incremental learning once we achieve the desired performance.

The reason behind this initial sampling is to learn an initial causal model and given that our sampling budget is limited, we maximize the information in the causal model iteratively following a Bayesian approach where at iteration t the current causal model is considered as prior and given the new sample it gets updated.

Stage-V: Estimate Performance Queries At this stage, given the learned causal performance model, UNICORN's inference engine estimates the user-specified queries using the mathematics of causal reasoning—do-calculus. Specifically, the causal inference engine provides a quantitative estimate for the identifiable queries on the current causal model and may return some queries as unidentifiable. It also determines what assumptions or new measurements are required to

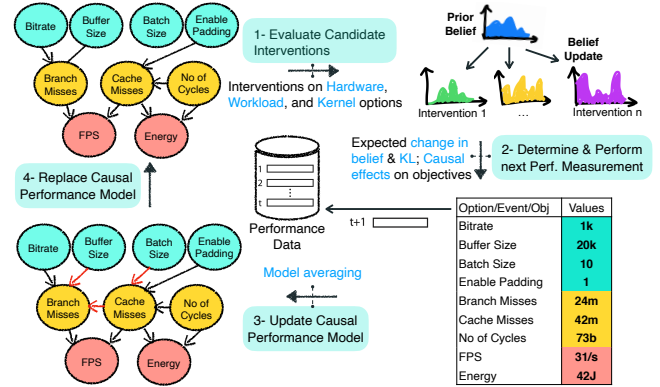


Figure 11. Causal model update.

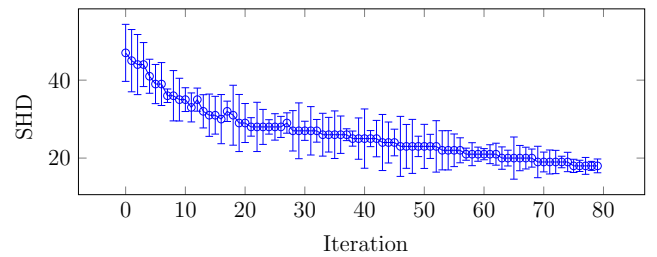


Figure 12. The structural hamming distance (SHD) between the learned causal model and ground truth model decreases as the algorithms measures more configuration samples.

answer the “unanswerable” questions, so, the user can decide to incorporate these new assumptions by defining more constraints or increase the sampling budgets.

Implementation. There are generic causal inference tools maintained by industry (e.g., Microsoft’s DoWhy [90], IBM’s CausalLib [91], Uber’s CausalML [100]), and academia (e.g., Ananke [13], Causal Fusion [12]); these tools only implement the core machinery in causal reasoning including different causal modeling, structure learning, inference, counterfactuals, and estimation tasks. We implemented UNICORN by integrating and building on top of (i) *Semopy* [89] for predictions with causal models, (ii) *Ananke* [13] for estimating the causal effects, (iii) *CausalML* [100] for counterfactual reasoning, *PyCausal* [84] for structure learning. In particular, we integrated the tools into a seamless pipeline to provide users with the following capabilities for performance modeling and analyses: (i) **Modeling**: specifying the causal variables and constraints needed for learning a correct causal performance model (see Section 2); (ii) **Analyses**: Iterative sampling, model learning and update, estimation of performance queries, and early stopping; and (iii) **User interface**: visualization, annotations, interactions with CPM models, and query estimations.

Problem [2]: For a real-time scene detection task, TX2 (faster platform) only processed 4 frames/sec whereas TX1 (slower platform) processed 17 frames/sec, i.e., the latency is 4× worse on TX2. Observed Latency (frames/sec): 4 FPS Expected Latency (frames/sec): 22-24 FPS (30-40% better)					
Configuration Options	UNICORN	SMAC	BugDoc	Forum	ACE [†]
CPU Cores	✓	✓	✓	✓	3%
CPU Frequency	✓	✓	✓	✓	6%
EMC Frequency	✓	✓	✓	✓	13%
GPU Frequency	✓	✓	✓	✓	22%
Scheduler Policy	✓	✓	✓	✓	✓
kernel.sched_rt_runtime_us	✓	✓	✓	✓	✓
kernel.sched_child_runs_first	✓	✓	✓	✓	✓
vm.dirty_background_ratio	✓	✓	✓	✓	✓
vm.dirty_ratio	✓	✓	✓	✓	✓
Drop Caches	✓	✓	✓	✓	✓
CUDA_STATIC	✓	✓	✓	✓	55%
vm.vfs_cache_pressure	✓	✓	✓	✓	✓
vm.swappiness	✓	✓	✓	✓	1%
Latency (TX2 frames/sec)	28	24	20	23	
Latency Gain (over TX1)	65%	42%	21%	39%	
Latency Gain (over default)	7×	6×	5×	5.75×	
Resolution time	22 mins	4 hrs	3.5 hrs	2 days	

Figure 13. Using UNICORN on a real-world performance issue.

4 Case Study

Prior to a systematic evaluation in Section ??, here, we show how UNICORN can enable performance debugging in a real-world scenario discussed in [2], where a developer migrated a real-time scene detection system from NVIDIA TX1 to a more powerful hardware, TX2. However, the developer, surprisingly, experienced 4× worse latency in the new environment (from 17 frames/sec in TX1 to 3 frames/sec in TX2). After two days of discussions, the performance issue was diagnosed with a misconfiguration—an incorrect setting of a compiler option and two hardware options. Here, we assess whether and how UNICORN could facilitate the performance debugging by comparing with (i) the fix suggested by NVIDIA in the forum, and two academic performance debugging approaches—BUGDOC [68] and SMAC [50].

Findings. Fig. 13 illustrates our findings. We find that:

- UNICORN could diagnose the root cause of the misconfiguration and recommends a fix within 24 minutes. Using the recommended configuration fixes from UNICORN, we achieved a throughput of 28 frames/sec (65% higher than TX1 and 7× higher than the fault). This, surprisingly, exceeds the developers’ initial expectation of 30 – 40% improvement.
- BugDoc (a diagnosis approach) has the least improvement compared to other approaches (21% improvement over TX1), while taking 3.5 hours to suggest the fix. BugDoc also changed several unrelated options (depicted by ✓) not endorsed by the domain experts.

- Using SMAC (an optimization approach), we aimed to find a configuration that achieves optimal throughput. However, after converging, SMAC recommended a configuration which achieved 24 frames/sec (42% better than TX1 and 6× better than the fault), however, could not outperform the configuration suggested by UNICORN and even took 4 hours (6× longer than UNICORN to converge). In addition, SMAC changed several unrelated options (✓ in Fig. 13).

Why UNICORN works better (and faster)? UNICORN discovers the misconfigurations by constructing a causal model (a simplified version of this is shown in Fig. 13). This causal model rules out irrelevant configuration options and focuses on the configurations that have the highest (direct or indirect) causal effect on latency, e.g., we found the root-cause CUDA STATIC in the causal graph which indirectly affects latency via context-switches (an intermediate system event); this is similar to other relevant configurations that indirectly affected latency (via energy consumption). Using counterfactual queries, UNICORN can reason about changes to configurations with the highest average causal effect (ACE) (last column in Fig. 13). The counterfactual reasoning occurs no additional measurements, significantly speeding up inference as shown in Fig. 13, UNICORN accurately finds all the configuration options recommended by the forum (depicted by ✓ in Fig. 13).

5 Evaluations

We examine the following by comparing UNICORN with state-of-the-art performance debugging and optimization approaches: (i) **Effectiveness** (Section 6) in terms of sample efficiency and performance gain. (ii) **Transferability** (Section 7) of learned models across environments, and (iii) **Scalability** (Section 8) to large-scale highly-configurable systems.

Systems. We selected six configurable systems including a video analytic pipeline, three deep learning-based systems (for image, speech, and NLP), a video encoder, and a database, see Table 1. We use heterogeneous deployment platforms, including NVIDIA TX1, TX2, and XAVIER, each having different resources (compute, memory) and microarchitectures. **Configuration.** We choose a wide range of configuration options and system events (see Table 1), following NVIDIA’s configuration guides/tutorials and other related work [39]. As opposed to prior work that only can support binary options due to scalability issues (e.g., [102, 103], we included options with binary, discrete, and continuous.

Ground truth. To ensure reliable and replicable results, following the common practice [23, 26, 56, 60], we measured 2000 samples for each 18 deployment settings (6 systems and 3 hardware) and repeated each measurement 5 times and recorded the median. We curated a ground truth of performance issues, called JETSON FAULTS, for each of the studied

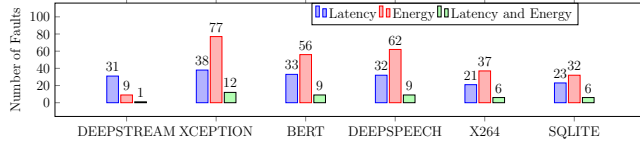


Figure 14. Distribution of 451 single-objective and 43 multi-objective non-functional faults across different software systems used in our study.

Table 1. Overview of the subject systems used in our study. More details about the configuration options and system events for each system are found in Table 4 - 10 in Appendix.

System	Workload	C	O	S	H	W	P
DEEPSTREAM [4]	Video analytics pipeline for detection and tracking from 8 camera streams.	2461	53	288	2	1	2
XCEPTION [18]	Image recognition system to classify 5000/5000 test images from CIFAR10.	6443	28	19	3	3	3
DEEPSPEECH [43]	Speech-to-text from 0.5/1932 hours of Common Voice Corpus 5.1 (English) data.	6112	28	19	3	1	3
BERT [25]	NLP system for sentiment analysis of 1000/25000 test reviews from IMDb.	6188	28	19	3	1	3
X264 [6]	Encodes a 20 second 11.2 MB video of resolution 1920 x 1080 from UGC.	17248	32	19	3	1	3
SQLITE [5]	Database engine for sequential & batch & random reads, writes, deletions.	15680	242	288	3	3	3

* C: Configurations, O: Options, S: System Events, H: Hardware, W: Workload, P: Objectives

software and hardware systems using the measured ground truth data. By definition, non-functional faults are located in the tail of performance distributions [37, 62]. We, therefore, selected and labeled configurations that are worse than the 99th percentile as ‘faulty.’ Fig. 14 shows the total 494 NF-FAULTS discovered across different software. Out of these 494 NF-FAULTS, 43 are faults with multiple types (both energy and latency). Of all the 451 single-objective and 43 multi-objective NF-FAULTS discovered in this study, only 2 faults had a single root cause, 411 faults had five or more root causes, and 81 remaining faults had two to four root causes. *Initial samples:* 25–10% of the total sampling budget.

Baselines. We evaluate UNICORN for two performance tasks, including **Task 1. Performance Debugging and Repair** and **Task 2. Performance Optimization**. We compare UNICORN against state-of-the-art, including **CBI** [95], a feature selection algorithm for fixing performance issues; **DD** [8], a delta debugging technique, that minimizes the difference between a pair of configurations; **EnCore** [110] learns to debug from correlational information about misconfigurations; **BugDoc** [68] infers the root causes and derive succinct explanations of failures using decision trees; **SMAC** [50], a sequential model-based auto-tuning approach; and **PESMO** [45], a multi-objective Bayesian optimization approach.

Evaluation Metrics. (i) **Recall**, the percentage of true root-causes that are correctly predicted, (ii) **Precision**, the percentage of true root-causes among the predicted ones, (iii) **Accuracy**, weighted Jaccard similarity between the predicted and true root-causes, where the weight vector was derived based on the average causal effect (see (1)) of options to performance based on the ground-truth CPM. For example, if A is the recommended configuration by an approach and B is the configuration that fixes the performance issue in the ground truth, we measure $accuracy = \frac{\sum_{ACE} (A \cap B)}{\sum_{ACE} (A \cup B)}$. The key intuition is that an ideal causal model underlying the system should identify the most important options that affect performance objectives. In other words, an ideal causal model should provide recommendations for changing the values of options that has the highest average causal effects on system performance. (iv) **Gain**, percentage improvement of suggested fix over the observed fault— $\Delta_{gain} = \frac{NFP_{FAULT} - NFP_{NOFAULT}}{NFP_{FAULT}} \times 100$, where NFP_{FAULT} the observed faulty performance and $NFP_{NO FAULT}$ is the performance of suggested fix. (v) **Error**: Single objective: the absolute difference between the best suggested and the optimal in ground truth; Multi-objective: hypervolume error [113]. (vi) **Time**, overhead of an approach in terms of wallclock time in hours to converge and suggest a fix.

6 Effectiveness and Sample Efficiency

Setting: We only show the partial results, however, our results generalize to all evaluated settings. *Debugging:* latency faults in TX2 and energy faults in XAVIER. *Optimization (single-objective):* comparison with SMAC in TX2 for XCEPTION for both latency and energy. *Optimization (multi-objective):* comparison with PESMO in TX2. We repeat the entire debugging and optimization tasks 3 times.

Results (debugging). Tables 13a and 14 shows UNICORN significantly outperforms correlation-based methods in all cases. For example, in DEEPSTREAM on TX2, UNICORN achieves 6% more accuracy, 12% more precision, and 10% more recall compared to the next best method, BugDoc. We observed latency gains as high as 88% (9% more than BugDoc) on TX2 and energy gain of 86% (9% more than BugDoc) on XAVIER for XCEPTION. We observe similar trends in energy faults and multi-objective faults. The results confirm that UNICORN can recommend repairs for faults that significantly improve latency and energy usage. Applying the changes to the configurations recommended by UNICORN increases the performance drastically.

Fig. 16a and Fig. 16b demonstrate the sample efficiency results for different systems. We observe that for both latency and energy, faults UNICORN achieved significantly higher gains with significantly less number of samples. For XCEPTION, UNICORN required a 8x lower number of samples to obtain 32% higher gain than DD. The higher gain in UNICORN with a relatively lower number of samples in comparison

Table 2. Efficiency of UNICORN compared to other approaches. Cells highlighted in **blue** indicate improvement over faults.**(a)** Single objective performance fault in latency and energy consumption.

			Accuracy					Precision					Recall					Gain					Time [†]	
			UNICORN	CBI	DD	ENCORE	BugDoc	UNICORN	CBI	DD	ENCORE	BugDoc	UNICORN	CBI	DD	ENCORE	BugDoc	UNICORN	CBI	DD	ENCORE	BugDoc	UNICORN	Others
TX2	Latency	DEEPSTREAM	87	61	62	65	81	83	66	59	60	71	80	61	65	60	70	88	66	67	68	79	0.8	4
		XCEPTION	86	53	42	62	65	86	67	61	63	67	83	64	68	69	62	82	48	42	57	59	0.6	4
		BERT	81	56	59	60	57	76	57	55	61	73	71	74	68	67	65	74	54	59	62	58	0.4	4
		DEEPSPEECH	81	61	59	60	72	76	58	69	61	71	81	73	61	63	69	76	59	53	55	66	0.7	4
		x264	83	59	63	62	62	82	69	58	65	66	78	64	67	63	72	85	69	72	68	71	1.4	4
XAVIER	Energy	DEEPSTREAM	91	81	79	77	87	81	61	62	64	73	85	63	61	62	75	86	68	62	61	78	0.7	4
		XCEPTION	84	66	63	63	81	78	56	58	66	65	80	69	55	63	68	83	59	50	51	62	0.4	4
		BERT	66	59	53	63	72	70	62	64	64	65	79	61	54	63	66	62	49	36	49	53	0.5	4
		DEEPSPEECH	73	68	63	72	71	75	55	59	54	68	78	53	52	59	71	78	64	48	65	63	1.2	4
		x264	77	71	70	74	74	83	63	53	61	66	78	67	53	54	72	87	73	71	76	76	0.3	4

(b) Multi-objective non-functional faults in Energy, Latency.

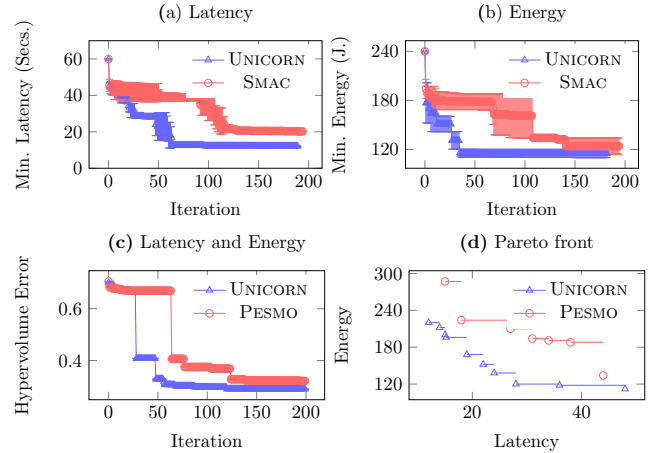
			Accuracy				Precision				Recall				Gain (Latency)				Gain (Energy)				Time [†]	
			UNICORN	CBI	ENCORE	BugDoc	UNICORN	CBI	ENCORE	BugDoc	UNICORN	CBI	ENCORE	BugDoc	UNICORN	CBI	ENCORE	BugDoc	UNICORN	CBI	ENCORE	BugDoc	UNICORN	Others
Energy + Latency	Latency	XCEPTION	89	76	81	79	77	53	54	62	81	59	59	62	84	53	61	65	75	38	46	44	0.9	4
		BERT	71	72	73	71	77	42	56	63	79	59	62	65	84	53	59	61	67	41	27	48	0.5	4
		DEEPSPEECH	86	69	71	72	80	44	53	62	81	51	59	64	88	55	55	62	77	43	43	41	1.1	4
		x264	85	73	83	81	83	50	54	67	80	63	62	61	75	62	64	66	76	64	66	64	1	4

[†] Wallclock time in hours

to correlation-based methods indicates that UNICORN causal reasoning is more effective in guiding the search in the objective space. UNICORN does not waste budget with evaluating configurations with lower causal effects and finds a fix faster.

UNICORN can resolve misconfiguration faults significantly faster than correlation-based approaches. In Tables 13a and 14, the last two columns indicate the time taken (in hours) by each approach to diagnosing the root cause. For all methods, we set a maximum budget of 4 hours. We find that, while other approaches use the entire budget to diagnose and resolve the faults, UNICORN can do so significantly faster, e.g., UNICORN is 13× faster in diagnosing and resolving faults in energy usage for x264 deployed on XAVIER and 10× faster for latency faults for BERT on TX2.

Results (optimization). Fig. 15 (a) and Fig. 15 (b) demonstrate the single-objective optimization results—UNICORN finds configurations with optimal latency and energy for both cases. Fig. 15 (a) illustrates that the optimal configuration discovered by UNICORN has 43% lower latency (12 seconds) than that of SMAC (21 seconds). Here, UNICORN reaches near-optimal configuration by only exhausting one-third of the entire budget. In Fig. 15 (b), the optimal configuration discovered by UNICORN and SMAC had almost the same energy, but UNICORN reached this optimal configuration 4x faster than SMAC. In both single-objective optimizations, the iterative variation of UNICORN is less than SMAC—i.e., UNICORN finds more stable configurations. Fig. 15 (c) compares UNICORN with PESMO to optimize both latency and

**Figure 15.** UNICORN vs. optimization with SMAC and PESMO.

energy in TX2 (for image recognition). Here, UNICORN has 12% lower hypervolume error than PESMO and reaches the same level of hypervolume error of PESMO 4x times faster. Fig. 15 (d) illustrates the optimal Pareto front obtained by UNICORN and PESMO. The Pareto front by UNICORN has higher coverage, as it discovered a higher number of Pareto optimal configurations with lower energy and latency value than PESMO.

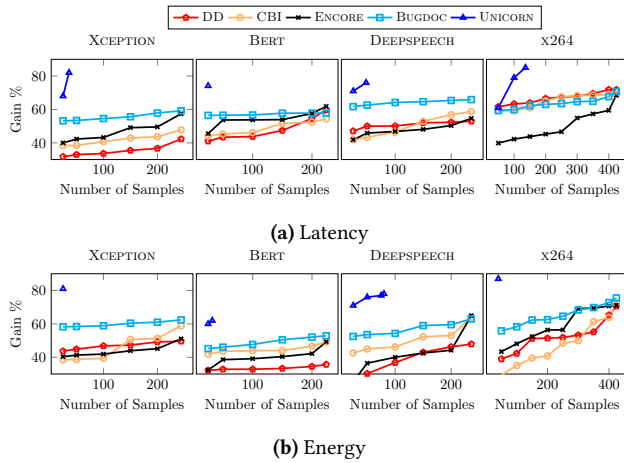


Figure 16. UNICORN has significantly higher sampling efficiency than other baselines in debugging non-functional faults.

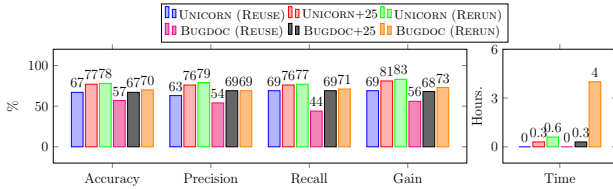


Figure 17. Accuracy, Precision, Recall, and Gain of debugging non-functional faults (XAVIER to TX2).

7 Transferability

Setting. We reuse the CPM constructed from a source environment, e.g., TX1, to resolve a non-functional fault in a target environment, e.g., XAVIER. We evaluated UNICORN for debugging energy faults for XCEPTION and used XAVIER as the source and TX2 as the target, since they have different microarchitectures, expecting to see large differences in their performance behaviors. We only compared with BUGDOC as it discovered fixes with higher energy gain in XAVIER than other correlation-based baseline methods (see Table 13a). We compared UNICORN and BUGDOC in the following scenarios: (I) REUSE: reusing the recommended configurations from Source to Target, (II) +25: reusing the performance models (i.e., causal model and decision tree) learned in Source and finetuned the models with 25 new samples in Target, and (III) RERUN: we rerun UNICORN and BUGDOC from scratch to resolve energy faults in Target. For optimization tasks, we use three larger additional XCEPTION workloads: 10000 (10k), 20000 (20k), and 50000 (50k) test images (previous experiments used 5000 (5k) test images). We evaluated three variants of SMAC and UNICORN: (1) SMAC and UNICORN (REUSE), where we *reuse* the near-optimum found with a 5K tests image on the larger workloads; (2) SMAC +10% and UNICORN +10%, where we rerun with 10% budget in target and update the CPM with 10% additional budget; and (3) SMAC

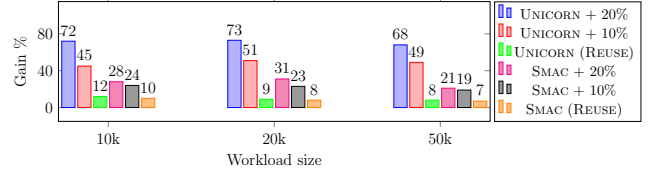


Figure 18. UNICORN finds configurations with higher gain when workloads are changed for performance optimization task.

+20% and UNICORN +20%, where we rerun with 20% budget in target and update the model with 20% additional budget. **Results.** Fig. 17 indicates the results in resolving energy faults in TX2. We observe that UNICORN +25 obtains 8% more accuracy, 7% more precision, 5% more recall and 8% more gain than BUGDOC (RERUN). Here, BUGDOC takes significantly longer than UNICORN, i.e., BUGDOC (RERUN) exceeds the 4-hour budget in whereas UNICORN takes at most 20 minutes to fix the energy faults. We have to rerun BUGDOC every time the hardware changes, and this limits its practical usability. In contrast, UNICORN incrementally updates the internal causal model with new samples from the newer hardware to learn new relationships. Therefore, it is less sensitive and much faster. Since UNICORN uses causal models to infer candidate fixes using only the available observational data, it tends to be much faster than BUGDOC. We also observe that with little updates, UNICORN +25 (~20 minutes) achieves a similar performance of UNICORN (RERUN) (~36 minutes). Since the causal mechanisms are sparse, the CPM from XAVIER in UNICORN quickly reaches a fixed structure in TX2 using incremental learning by judiciously evaluating the most promising fixes until the fault is resolved.

Our experimental results demonstrate that UNICORN performs better than the two variants of three SMAC (c.f. Fig. 18). SMAC (REUSE) performs the worst when the workload changes. With 10K images, reusing the near-optimal configuration from 5K images results in a latency gain of 10%, compared to 12% with UNICORN in comparison with the default configuration. We observe that UNICORN + 20% achieves 44%, 42%, and 47% higher gain than SMAC + 20% for workload sizes of 10k, 20k, and 50k images, respectively.

8 Scalability

Setting. We evaluated UNICORN for scalability with SQLITE (large configuration space) and DEEPSTREAM (large composed system). In SQLITE, we conducted the evaluation in three scenarios: (a) selecting the most relevant software, hardware options and events (34 configuration options and 19 system events), (b) selecting all modifiable software and hardware options and system events (242 configuration options and 19 events), and (c) selecting not only all modifiable software and hardware options and system events but also intermediate tracepoint events (242 configuration

options and 288 events). In DEEPSTREAM, there are two scenarios: (a) 53 configuration options and 19 system events, and (b) 53 configuration options and 288 events when we select all modifiable software and hardware options, and system/tracepoint events.

Table 3. Scalability for SQLite and DEEPSTREAM on XAVIER.

System	Configs	Events	Paths	Queries	Degree	Gain (%)	Time/Fault (in sec.)		
							Discovery	Query Eval	Total
SQLITE	34	19	32	191	3.6	93	9	14	291
	242	19	111	2234	1.9	94	57	129	1345
	242	288	441	22372	1.6	92	111	854	5312
DEEPSTREAM	53	19	43	497	3.1	86	16	32	1509
	53	288	219	5008	2.3	85	97	168	3113

Results. In large systems, there are significantly more causal paths and therefore, causal learning and estimations of queries take more time. The results in Table 3 indicate that UNICORN can scale to a much larger configuration space without an exponential increase in runtime for any of the intermediate stages. This can be attributed to the sparsity of the causal graph (average degree of a node for SQLITE in Table 3 is at most 3.6, and it reduces to 1.6 when the number of configurations increase and reduces from 3.1 to 2.3 in DEEPSTREAM when systems events are increased).

9 Related Work

Performance Faults in Configurable Systems. Previous empirical studies have shown that a majority of performance issues are due to misconfigurations [41], with severe consequences in production environments [69, 98], and configuration options that cause such performance faults force the users to tune the systems themselves [112]. Previous works have used static and dynamic program analysis to identify the influence of configuration options on performance [67, 102, 103] and to detect and diagnose misconfigurations [9, 10, 109, 111]. Unlike UNICORN, none of the white-box analysis approaches target configuration space across the system stack, where it limits their applicability in identifying the true causes of a performance fault.

Statistical and Model-based Debugging. Debugging approaches such as STATISTICAL DEBUGGING [95], HOLMES [17], XTREE [66], BUGDOC [68], ENCORE [68], REX [70], and PERFLEARNER [42] have been proposed to detect root causes of system faults. These methods make use of statistical diagnosis and pattern mining to rank the probable causes based on their likelihood of being the root causes of faults. However, these approaches may produce correlated predicates that lead to incorrect explanations.

Causal Testing and Profiling. Causal inference has been used for fault localization [11, 31], resource allocations in

cloud systems [33], and causal effect estimation for advertisement recommendation systems [15]. More recently, AID [30] detects root causes of an intermittent software failure using fault injection as interventions. CAUSAL TESTING [59] modifies the system inputs to observe behavioral changes and utilizes counterfactual reasoning to find the root causes of bugs. Causal profiling approaches like CoZ [24] points developers where optimizations will improve performance and quantifies their potential impact. Causal inference methods like X-RAY [9] and CONFAID [10] had previously been applied to analyze program failures. All approaches above are either orthogonal or complimentary to UNICORN, mostly they focus on functional bugs (e.g., CAUSAL TESTING) or if they are performance related, they are not configuration-aware (e.g., CoZ).

10 Conclusion

Modern computer systems are highly-configurable with thousands of interacting configurations with a complex performance behavior. Misconfigurations in these systems can elicit complex interactions between software and hardware configuration options, resulting in non-functional faults. We propose UNICORN, a novel approach for diagnostics that learns and exploits the system’s causal structure consists of configuration options, system events, and performance metrics. Our evaluation shows that UNICORN effectively and quickly diagnoses the root cause of non-functional faults and recommends high-quality repairs to mitigate these faults. We also show that the learned causal performance model is transferable across different workload and deployment environments. Finally, we demonstrate the scalability of UNICORN scales to large systems consists of 500 options and several trillion potential configurations.

Acknowledgements

This work has been supported in part by NASA (Awards 80NSSC20K1720 and 521418-SC) and NSF (Awards 2007202, 2107463, and 2038080), Google, and Chameleon Cloud. We are grateful to all who provided feedback on this work, including Christian Kästner, Sven Apel, Yuriy Brun, Emery Berger, Tianyin Xu, Vivek Nair, Jianhai Su, Miguel Velez, Tobias Dürschmid, and the anonymous EuroSys’22 (as well as EuroSys’21 and FSE’21) reviewers.

References

- [1] Slow image classification with tensorflow on TX2. In NVIDIA developer forums: <https://forums.developer.nvidia.com/t/54307>, October 2017.
- [2] Cuda performance issue on TX2. In NVIDIA developer forums: <https://forums.developer.nvidia.com/t/50477>, June 2020.
- [3] General performance problems. In NVIDIA developer forums: <https://forums.developer.nvidia.com/t/111704>, February 2020.
- [4] Nvidia deepstream sdk. <https://developer.nvidia.com/deepstream-sdk>, 2021.
- [5] Sqlite database engine. <https://www.sqlite.org/index.html>, 2021.

- [6] X264 video encoder. <https://www.videolan.org/developers/x264.html>, 2021.
- [7] ALCOCER, J. P. S., BERGEL, A., DUCASSE, S., AND DENKER, M. Performance evolution blueprint: Understanding the impact of software evolution on performance. In *Proc. of Working Conference on Software Visualization (VISSOFT)* (2013), IEEE, pp. 1–9.
- [8] ARTHO, C. Iterative delta debugging. *International Journal on Software Tools for Technology Transfer* 13, 3 (2011), 223–246.
- [9] ATTARIYAN, M., CHOW, M., AND FLINN, J. X-ray: Automating root-cause diagnosis of performance anomalies in production software. In *Presented as part of the 10th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 12)* (2012), pp. 307–320.
- [10] ATTARIYAN, M., AND FLINN, J. Automating configuration troubleshooting with dynamic information flow analysis. In *OSDI* (2010), vol. 10, pp. 1–14.
- [11] BAAH, G. K., PODGURSKI, A., AND HARROLD, M. J. Causal inference for statistical fault localization. In *Proceedings of the 19th international symposium on Software testing and analysis* (2010), pp. 73–84.
- [12] BAREINBOIM, E. Causal fusion. <https://www.causalfusion.net/>, 2021.
- [13] BHATTACHARYA, R., NABI, R., AND SHPITSER, I. Semiparametric inference for causal effects in graphical models with hidden variables. *arXiv preprint arXiv:2003.12659* (2020).
- [14] BLAMES 'FAULTY CONFIGURATION CHANGE' FOR NEARLY SIX-HOUR OUTAGE, F. <https://t.ly/Dy1s>.
- [15] BOTTOU, L., PETERS, J., QUIÑONERO-CANDELA, J., CHARLES, D. X., CHICKERING, D. M., PORTUGALY, E., RAY, D., SIMARD, P., AND SNELSON, E. Counterfactual reasoning and learning systems: The example of computational advertising. *The Journal of Machine Learning Research* 14, 1 (2013), 3207–3260.
- [16] BRYANT, R. E., DAVID RICHARD, O., AND DAVID RICHARD, O. *Computer systems: a programmer's perspective*, vol. 2. 2003.
- [17] CHILIMBI, T. M., LIBLIT, B., MEHRA, K., NORI, A. V., AND VASWANI, K. Holmes: Effective statistical debugging via efficient path profiling. In *2009 IEEE 31st International Conference on Software Engineering* (2009), IEEE, pp. 34–44.
- [18] CHOLLET, F. Xception: Deep learning with depthwise separable convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (2017), pp. 1251–1258.
- [19] CLOUD OUTAGE WAS TRIGGERED BY CONFIGURATION ERROR, A. <https://www.computerworld.com/article/2508335/amazon-cloud-outage-was-triggered-by-configuration-error.html>.
- [20] COLOMBO, D., AND MAATHUIS, M. H. Order-independent constraint-based causal structure learning. *The Journal of Machine Learning Research* 15, 1 (2014), 3741–3782.
- [21] COLOMBO, D., MAATHUIS, M. H., KALISCH, M., AND RICHARDSON, T. S. Learning high-dimensional directed acyclic graphs with latent and selection variables. *The Annals of Statistics* (2012), 294–321.
- [22] CONNELLY, L. M. Fisher's exact test. *Medsurg Nursing* 25, 1 (2016).
- [23] CURTSINGER, C., AND BERGER, E. D. Stabilizer: Statistically sound performance evaluation. *ACM SIGARCH Computer Architecture News* 41, 1 (2013), 219–228.
- [24] CURTSINGER, C., AND BERGER, E. D. Coz: Finding code that counts with causal profiling. In *Proceedings of the 25th Symposium on Operating Systems Principles* (2015), pp. 184–197.
- [25] DEVLIN, J., CHANG, M.-W., LEE, K., AND TOUTANOVA, K. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).
- [26] DING, Y., PERVAIZ, A., CARBIN, M., AND HOFFMANN, H. Generalizable and interpretable learning for configuration extrapolation. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (2021), pp. 728–740.
- [27] ELKHODARY, A., ESFAHANI, N., AND MALEK, S. Fusion: A framework for engineering self-tuning self-adaptive software systems. In *Proc. Int'l Symp. Foundations of Software Engineering (FSE)* (2010), ACM, pp. 7–16.
- [28] ESFAHANI, N., ELKHODARY, A., AND MALEK, S. A learning-based framework for engineering feature-oriented self-adaptive software systems. *IEEE Trans. Softw. Eng. (TSE)* 39, 11 (2013), 1467–1493.
- [29] FARAHMAND, S., O'CONNOR, C., MACOSKA, J. A., AND ZARRINGHALAM, K. Causal inference engine: a platform for directional gene set enrichment analysis and inference of active transcriptional regulators. *Nucleic acids research* 47, 22 (2019), 11563–11573.
- [30] FARIHA, A., NATH, S., AND MELIOU, A. Causality-guided adaptive interventional debugging. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data* (2020), pp. 431–446.
- [31] FEYZI, F., AND PARSIA, S. Infrence: effective fault localization based on information-theoretic analysis and statistical causal inference. *Frontiers of Computer Science* 13, 4 (2019), 735–759.
- [32] FILIERI, A., HOFFMANN, H., AND MAGGIO, M. Automated multi-objective control for self-adaptive software design. In *Proc. Int'l Symp. Foundations of Software Engineering (FSE)* (2015), ACM, pp. 13–24.
- [33] GEIGER, P., CARATA, L., AND SCHÖLKOPF, B. Causal models for debugging and control in cloud computing. *arXiv preprint arXiv 1603* (2016).
- [34] GLYMOUR, C., ZHANG, K., AND SPIRITES, P. Review of causal discovery methods based on graphical models. *Frontiers in genetics* 10 (2019), 524.
- [35] GREBHACH, A., SIEGMUND, N., AND APEL, S. Predicting performance of software configurations: There is no silver bullet. *arXiv preprint arXiv:1911.12643* (2019).
- [36] GREBHACH, A., SIEGMUND, N., KÖSTLER, H., AND APEL, S. Performance prediction of multigrid-solver configurations. In *Software for Exascale Computing-SPPEXA 2013-2015*. Springer, 2016, pp. 69–88.
- [37] GUNAWI, H. S., ET AL. Fail-slow at scale: Evidence of hardware performance faults in large production systems. *ACM Transactions on Storage (TOS)* 14, 3 (2018), 1–26.
- [38] GUO, J., CZARNECKI, K., APEL, S., SIEGMUND, N., AND WASOWSKI, A. Variability-aware performance prediction: A statistical learning approach. In *Proc. Int'l Conf. Automated Software Engineering (ASE)* (2013), IEEE.
- [39] HALAWA, H., ABDELHAFEZ, H. A., BOKTOR, A., AND RIPEANU, M. NVIDIA jetson platform characterization. *Lect. Notes Comput. Sci. (including Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinformatics)* 10417 LNCS (2017), 92–105.
- [40] HALIN, A., NUTTINCK, A., ACHER, M., DEVROEY, X., PERROUIN, G., AND BAUDRY, B. Test them all, is it worth it? assessing configuration sampling on the jhipster web development stack. *Empirical Software Engineering* 24, 2 (2019), 674–717.
- [41] HAN, X., AND YU, T. An empirical study on performance bugs for highly configurable software systems. In *Proceedings of the 10th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement* (2016).
- [42] HAN, X., YU, T., AND LO, D. Perflearner: learning from bug reports to understand and generate performance test frames. In *2018 33rd IEEE/ACM International Conference on Automated Software Engineering (ASE)* (2018).
- [43] HANNUN, A., CASE, C., CASPER, J., CATANZARO, B., DIAMOS, G., ELSSEN, E., PRENGER, R., SATHEESH, S., SENGUPTA, S., COATES, A., ET AL. Deep speech: Scaling up end-to-end speech recognition. *arXiv preprint arXiv:1412.5567* (2014).
- [44] HENARD, C., PAPADAKIS, M., HARMAN, M., AND LE TRAON, Y. Combining multi-objective search and constraint solving for configuring large software product lines. In *Proc. Int'l Conf. Software Engineering (ICSE)* (2015), IEEE, pp. 517–528.
- [45] HERNÁNDEZ-LOBATO, D., HERNÁNDEZ-LOBATO, J., SHAH, A., AND ADAMS, R. Predictive entropy search for multi-objective bayesian optimization. In *International Conference on Machine Learning* (2016),

- pp. 1492–1501.
- [46] HOFFMANN, H., SIDIROGLOU, S., CARBIN, M., MISAILOVIC, S., AGARWAL, A., AND RINARD, M. Dynamic knobs for responsive power-aware computing. In *In Proc. of Int'l Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)* (2011).
 - [47] HOOS, H. H. Automated algorithm configuration and parameter tuning. In *Autonomous search*. Springer, 2011, pp. 37–71.
 - [48] HOOS, H. H. Programming by optimization. *Communications of the ACM* 55, 2 (2012), 70–80.
 - [49] HU, Y., HUANG, G., AND HUANG, P. Automated reasoning and detection of specious configuration in large systems with symbolic execution. In *14th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 20)* (2020), pp. 719–734.
 - [50] HUTTER, F., HOOS, H. H., AND LEYTON-BROWN, K. Sequential model-based optimization for general algorithm configuration. In *International conference on learning and intelligent optimization* (2011), Springer, pp. 507–523.
 - [51] IQBAL, M. S., KOTTHOFF, L., AND JAMSHIDI, P. Transfer Learning for Performance Modeling of Deep Neural Network Systems. In *USENIX Conference on Operational Machine Learning* (Santa Clara, CA, 2019), USENIX Association.
 - [52] IQBAL, M. S., SU, J., KOTTHOFF, L., AND JAMSHIDI, P. Flexibo: Cost-aware multi-objective optimization of deep neural networks. *arXiv preprint arXiv:2001.06588* (2020).
 - [53] JAMSHIDI, P., AND CASALE, G. An uncertainty-aware approach to optimal configuration of stream processing systems. In *Proc. Int'l Symp. on Modeling, Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS)* (2016), IEEE.
 - [54] JAMSHIDI, P., AND CASALE, G. An uncertainty-aware approach to optimal configuration of stream processing systems. In *2016 IEEE 24th International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS)* (2016), IEEE, pp. 39–48.
 - [55] JAMSHIDI, P., GHAFARI, M., AHMAD, A., AND PAHL, C. A framework for classifying and comparing architecture-centric software evolution research. In *Proc. of European Conference on Software Maintenance and Reengineering (CSMR)* (2013), IEEE, pp. 305–314.
 - [56] JAMSHIDI, P., SIEGMUND, N., VELEZ, M., KÄSTNER, C., PATEL, A., AND AGARWAL, Y. Transfer learning for performance modeling of configurable systems: An exploratory analysis. In *Proc. Int'l Conf. Automated Software Engineering (ASE)* (2017), ACM.
 - [57] JAMSHIDI, P., VELEZ, M., KÄSTNER, C., AND SIEGMUND, N. Learning to sample: Exploiting similarities across environments to learn performance models for configurable systems. In *Proc. Int'l Symp. Foundations of Software Engineering (FSE)* (2018), ACM.
 - [58] JAMSHIDI, P., VELEZ, M., KÄSTNER, C., SIEGMUND, N., AND KAWTHEKAR, P. Transfer learning for improving model predictions in highly configurable software. In *Proc. Int'l Symp. Soft. Engineering for Adaptive and Self-Managing Systems (SEAMS)* (2017), IEEE.
 - [59] JOHNSON, B., BRUN, Y., AND MELIOU, A. Causal testing: Understanding defects' root causes. In *Proceedings of the 2020 International Conference on Software Engineering* (2020).
 - [60] KALTENECKER, C., GREBHAIN, A., SIEGMUND, N., AND APEL, S. The interplay of sampling and machine learning for software performance prediction. *IEEE Software* (2020).
 - [61] KAWTHEKAR, P., AND KÄSTNER, C. Sensitivity analysis for building evolving and & adaptive robotic software. In *Proceedings of the IJCAI Workshop on Autonomous Mobile Service Robots (WSR)* (7 2016).
 - [62] KLEPPMANN, M. *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. " O'Reilly Media, Inc.", 2017.
 - [63] KOCAOGLU, M., DIMAKIS, A. G., VISHWANATH, S., AND HASSIBI, B. Entropic causal inference. In *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence* (2017), p. 1156–1162.
 - [64] KOCAOGLU, M., SHAKKOTTAI, S., DIMAKIS, A., CARAMANIS, C., AND VISHWANATH, S. Applications of Common Entropy for Causal Inference.
 - [65] KOLESNIKOV, S., SIEGMUND, N., KÄSTNER, C., GREBHAIN, A., AND APEL, S. Tradeoffs in modeling performance of highly configurable software systems. *Software & Systems Modeling* 18, 3 (2019), 2265–2283.
 - [66] KRISHNA, R., MENZIES, T., AND LAYMAN, L. Less is more: Minimizing code reorganization using xtree. *Information and Software Technology* 88 (2017), 53–66.
 - [67] LI, C., WANG, S., HOFFMANN, H., AND LU, S. Statically inferring performance properties of software configurations. In *Proceedings of the Fifteenth European Conference on Computer Systems* (2020), pp. 1–16.
 - [68] LOURENÇO, R., FREIRE, J., AND SHASHA, D. Bugdoc: A system for debugging computational pipelines. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data* (2020), pp. 2733–2736.
 - [69] MAURER, B. Fail at scale: Reliability in the face of rapid change. *Queue* 13, 8 (2015), 30–46.
 - [70] MEHTA, S., BHAGWAN, R., KUMAR, R., BANSAL, C., MADDILA, C., ASHOK, B., ASTHANA, S., BIRD, C., AND KUMAR, A. Rex: Preventing bugs and misconfiguration in large services using correlated change analysis. In *17th {USENIX} Symposium on Networked Systems Design and Implementation* (2020).
 - [71] MOLYNEAUX, I. The art of application performance testing: Help for programmers and quality assurance.[sl]:" o'reilly media, 2009.
 - [72] MÜHLBAUER, S., APEL, S., AND SIEGMUND, N. Accurate modeling of performance histories for evolving software systems. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (2019), IEEE, pp. 640–652.
 - [73] MURASHKIN, A., ANTKIEWICZ, M., RAYSIDE, D., AND CZARNECKI, K. Visualization and exploration of optimal variants in product line engineering. In *Proc. Int'l Software Product Line Conference (SPLC)* (2013), ACM, pp. 111–115.
 - [74] NAIR, V., MENZIES, T., SIEGMUND, N., AND APEL, S. Faster discovery of faster system configurations with spectral learning. *arXiv:1701.08106* (2017).
 - [75] NISTOR, A., CHANG, P.-C., RADOI, C., AND LU, S. Caramel: Detecting and fixing performance problems that have non-intrusive fixes. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering* (2015).
 - [76] NISTOR, A., JIANG, T., AND TAN, L. Discovering, reporting, and fixing performance bugs. In *10th working conference on mining software repositories* (2013).
 - [77] NOROUZI, M., FLEET, D. J., AND SALAKHUTDINOV, R. R. Hamming distance metric learning. In *Advances in neural information processing systems* (2012), pp. 1061–1069.
 - [78] OGARRIO, J. M., SPIRITES, P., AND RAMSEY, J. A hybrid causal search algorithm for latent variable models. In *Conference on Probabilistic Graphical Models* (2016), pp. 368–379.
 - [79] OLAECHEA, R., RAYSIDE, D., GUO, J., AND CZARNECKI, K. Comparison of exact and approximate multi-objective optimization for software product lines. In *Proc. Int'l Software Product Line Conference (SPLC)* (2014), ACM, pp. 92–101.
 - [80] PEARL, J. Graphical models for probabilistic and causal reasoning. *Quantified representation of uncertainty and imprecision* (1998), 367–389.
 - [81] PEARL, J. *Causality*. Cambridge university press, 2009.
 - [82] PEARL, J., AND MACKENZIE, D. *The book of why: the new science of cause and effect*. Basic Books, 2018.
 - [83] PEREIRA, J. A., MARTIN, H., ACHER, M., JÉZÉQUEL, J.-M., BOTTERWECK, G., AND VENTRESQUE, A. Learning software configuration spaces: A systematic literature review. *arXiv preprint arXiv:1906.03018* (2019).

- [84] PyCAUSAL. Pycasual. <https://github.com/bd2kcccd/py-causal>, 2021.
- [85] REDDY, C. M., AND NALINI, N. Fault tolerant cloud software systems using software configurations. In *2016 IEEE International Conference on Cloud Computing in Emerging Markets (CCEM)* (2016), IEEE, pp. 61–65.
- [86] SÁNCHEZ, A. B., DELGADO-PÉREZ, P., MEDINA-BULO, I., AND SEGURA, S. Tandem: A taxonomy and a dataset of real-world performance bugs. *IEEE Access* 8 (2020), 107214–107228.
- [87] SCHERRER, N., BILANIUK, O., ANNADANI, Y., GOYAL, A., SCHWAB, P., SCHÖLKOPF, B., MOZER, M. C., BENGIO, Y., BAUER, S., AND KE, N. R. Learning neural causal models with active interventions. *arXiv preprint arXiv:2109.02429* (2021).
- [88] SCHÖLKOPF, B., LOCATELLO, F., BAUER, S., KE, N. R., KALCHBRENNER, N., GOYAL, A., AND BENGIO, Y. Toward causal representation learning. *Proceedings of the IEEE* 109, 5 (2021), 612–634.
- [89] SEMOPY. Semopy. <https://semopy.com/>, 2021.
- [90] SHARMA, A., KICIMAN, E., ET AL. DoWhy: A Python package for causal inference. <https://github.com/microsoft/dowhy>, 2021.
- [91] SHIMONI, Y., KARAVANI, E., RAVID, S., BAK, P., NG, T. H., ALFORD, S. H., MEADE, D., AND GOLDSCHMIDT, Y. An evaluation toolkit to guide model selection and cohort definition in causal inference. *arXiv preprint arXiv:1906.00442* (2019).
- [92] SIEGMUND, N., GREBHACH, A., APEL, S., AND KÄSTNER, C. Performance-influence models for highly configurable systems. In *Proc. Europ. Software Engineering Conf. Foundations of Software Engineering (ESEC/FSE)* (August 2015), ACM, pp. 284–294.
- [93] SIEGMUND, N., GREBHACH, A., APEL, S., AND KÄSTNER, C. Performance-influence models for highly configurable systems. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering* (2015), pp. 284–294.
- [94] SIEGMUND, N., RUCKEL, N., AND SIEGMUND, J. Dimensions of software configuration: on the configuration context in modern software development. In *Proceedings of the 28th ESEC/FSE* (2020), pp. 338–349.
- [95] SONG, L., AND LU, S. Statistical debugging for real-world performance problems. *ACM SIGPLAN Notices* 49, 10 (2014), 561–578.
- [96] SPIRITES, P., GLYMOUR, C. N., SCHEINES, R., AND HECKERMAN, D. *Causation, prediction, and search*. MIT press, 2000.
- [97] STYLES, J., HOOS, H. H., AND MÜLLER, M. Automatically configuring algorithms for scaling performance. In *Learning and Intelligent Optimization*. Springer, 2012, pp. 205–219.
- [98] TANG, C., KOOBURAT, T., VENKATACHALAM, P., CHANDER, A., WEN, Z., NARAYANAN, A., DOWELL, P., AND KARL, R. Holistic configuration management at facebook. In *Proceedings of the 25th Symposium on Operating Systems Principles* (2015), pp. 328–343.
- [99] TSAKILTSIDIS, S., MIRANSKY, A., AND MAZZAWI, E. On automatic detection of performance bugs. In *2016 IEEE international symposium on software reliability engineering workshops (ISSREW)* (2016), IEEE, pp. 132–139.
- [100] UBER. Causal ml. <https://github.com/uber/causalml>, 2021.
- [101] VALOV, P., PETKOVICH, J.-C., GUO, J., FISCHMEISTER, S., AND CZARNECKI, K. Transferring performance prediction models across different hardware platforms. In *Proc. Int'l Conf. on Performance Engineering (ICPE)* (2017), ACM, pp. 39–50.
- [102] VELEZ, M., JAMSHIDI, P., SATTTLER, F., SIEGMUND, N., APEL, S., AND KÄSTNER, C. Configcrusher: Towards white-box performance analysis for configurable systems. *Automated Software Engineering* 27 (2020), 265–300.
- [103] VELEZ, M., JAMSHIDI, P., SIEGMUND, N., APEL, S., AND KÄSTNER, C. White-box analysis over machine learning: Modeling performance of configurable systems. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)* (2021), IEEE, pp. 1072–1084.
- [104] VELEZ, M., JAMSHIDI, P., SIEGMUND, N., APEL, S., AND KÄSTNER, C. On debugging the performance of configurable software systems: Developer needs and tailored tool support. In *2022 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)* (2022).
- [105] WANG, S., LI, C., HOFFMANN, H., LU, S., SENTOSA, W., AND KISTIJANTORO, A. I. Understanding and auto-adjusting performance-sensitive configurations. *ACM SIGPLAN Notices* 53, 2 (2018).
- [106] WHITAKER, A., COX, R. S., GRIBBLE, S. D., ET AL. Configuration debugging as search: Finding the needle in the haystack. In *OSDI* (2004), vol. 4, pp. 6–6.
- [107] WU, F., WEIMER, W., HARMAN, M., JIA, Y., AND KRINKE, J. Deep parameter optimisation. In *Proc. of the Annual Conference on Genetic and Evolutionary Computation* (2015), ACM, pp. 1375–1382.
- [108] XIA, K., LEE, K.-Z., BENGIO, Y., AND BAREINBOIM, E. The causal-neural connection: Expressiveness, learnability, and inference.
- [109] XU, T., JIN, X., HUANG, P., ZHOU, Y., LU, S., JIN, L., AND PASUPATHY, S. Early detection of configuration errors to reduce failure damage. *USENIX Association*, pp. 619–634.
- [110] ZHANG, J., RENGANARAYANA, L., ZHANG, X., GE, N., BALA, V., XU, T., AND ZHOU, Y. Encore: Exploiting system environment and correlation information for misconfiguration detection. In *Proceedings of the 19th international conference on Architectural support for programming languages and operating systems* (2014), pp. 687–700.
- [111] ZHANG, S., AND ERNST, M. D. Automated diagnosis of software configuration errors. In *2013 35th International Conference on Software Engineering* (2013).
- [112] ZHANG, Y., HE, H., LEGUNSEN, O., LI, S., DONG, W., AND XU, T. An evolutionary study of configuration design and implementation in cloud systems. In *Proceedings of International Conference on Software Engineering* (2021), ICSE'21.
- [113] ZITZLER, E., BROCKHOFF, D., AND THIELE, L. The hypervolume indicator revisited: On the design of pareto-compliant indicators via weighted integration. In *International Conference on Evolutionary Multi-Criterion Optimization* (2007), Springer, pp. 862–876.

11 Appendix

11.1 Causal Performance Modeling and Analyses: Motivating Scenarios (Additional details)

Fig. 19 and Fig. 20 present additional scenarios where performance influence models could produce incorrect explanations. The regression terms presented here incorrectly identify spurious correlations, whereas the causal model correctly identifies the cause-effect relationships.

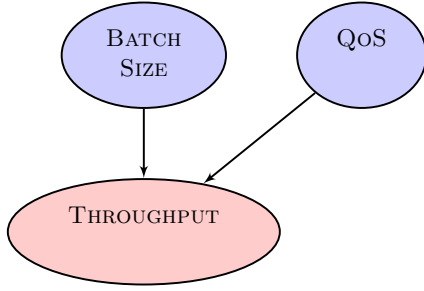


Figure 19. Regression model incorrectly identifies Batch Size and QoS are positively correlated with the term $0.08 \text{ Batch Size} \times \text{QoS}$ whereas they are unconditionally independent. Causal model correctly identifies the dependence (no causal connection) relationship between Batch Size and QoS (no arrow between Batch Size and QoS).

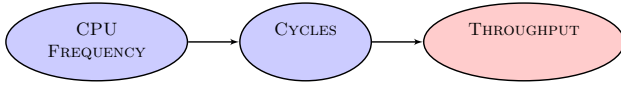


Figure 20. Causal model correctly identifies how CPU Frequency causally influences Throughput via Cycles whereas the regression $\text{Throughput} = 0.05 \times \text{CPU Frequency} \times \text{Cycles}$ identified incorrect interactions.

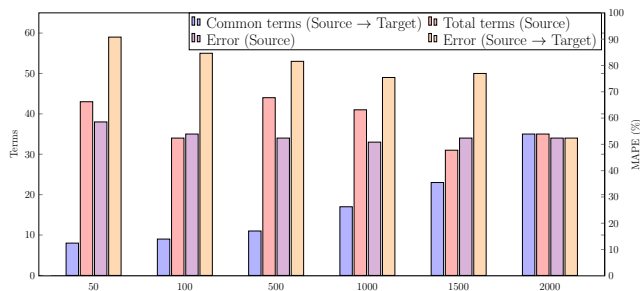


Figure 21. Performance influence models relying on correlational statistics are not stable as new samples are added and do not generalize well. Common terms refers to the individual predictors (i.e., options and interactions) in the performance models that are similar across environments.

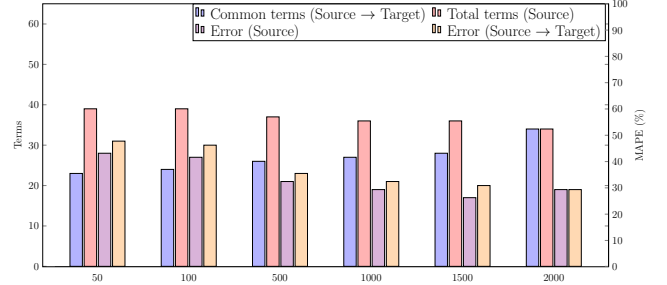


Figure 22. Causal performance models are relatively more stable as new samples are added and do generalize well.

Performance behavior of regression models for configurable systems varies when sample size varies. Fig. 21 shows the change of number of stable terms and error with different number of samples for building a performance influence models. Here, we vary the number of samples from 50 to 1500 to build a source regression model. We use sample size 2000 to build the target regression model. We observe that regression models cannot be reliably used in performance tasks, as they are sensitive to the number of training samples. The results indicate that this model classes as opposed to causal models cannot identify causal variables underlying system performance, so depending on the training sample, they try to find the best predictor to increase the prediction power with the i.i.d. assumption that does not hold in system performance. On the contrary, the number of stable predictor's variation is less in causal performance models and lead to better generalization as shown in Fig. 22. In addition to the number of stable predictors, the difference in error between source and target is negligible when compared to the performance regression models.

Extraction of predictor terms from the Causal Performance Model The constructed CPMs have performance objective nodes at the bottom (leaf nodes) and configuration options nodes at the top level. The intermediate levels are filled with the system events. To extract a causal term from the causal model, we backtrack starting from the performance objective until we reach a configuration option. If there are more than one path through a system event from performance objective to configuration options, we consider all possible interaction between those configuration options to calculate the number of causal terms.

11.2 UNICORN (Additional details)

Here, we explain some extra details in several stages in UNICORN to enable replicability of our approach.

Stage-II: Learn Causal Performance Model

In this section, we describe the edge orientation principles used in UNICORN.

Orienting undirected causal links. We orient undirected edges using prescribed edge orientation rules [20, 21, 34, 78,

96] to produce a *partial ancestral graph* (or PAG). A PAG contains the following types of (partially) directed edges:

- $X \longrightarrow Y$ indicating that vertex X causes Y .
- $X \longleftrightarrow Y$ which indicates that there are unmeasured confounders between vertices X and Y .

In addition, a PAG produces two types of partially directed edges:

- $X \circ \longrightarrow Y$ indicating that either X causes Y , or that there are *unmeasured confounders* that cause both X and Y .
- $X \circ \circ Y$ which indicates that either: (a) vertices X causes Y , or (b) vertex Y causes X , or (c) there are *unmeasured confounders* that cause both X and Y .

In the last two cases, the circle ($\circ$) indicates that there is an ambiguity in the edge type. In other words, given the current observational data, the circle can indicate an arrowhead ($\longrightarrow$) or no arrow head ($-$), i.e., for $X \circ \circ Y$, all three of $X \longrightarrow Y$, $Y \longrightarrow X$, and $X \longleftrightarrow Y$ might be compatible with current data, i.e., the current data could be faithful to each of these statistically equivalent causal graphs inducing the same conditional independence relationships.

Resolving partially directed edges. For subsequent analyses over the causal graph, the PAG obtained must be fully resolved (directed with no $\circ$ ended edges) in order to generate an ADMG, i.e., we must fully orient partially directed edges by replacing the circles in $\circ \longrightarrow$ and $\circ \circ$ with the correct edge direction. We use the information-theoretic approach using entropy proposed in [63, 64] to discover the true causal direction between two variables. Entropic causal discovery is inspired by Occam's razor, and the key intuition is that, among the possible orientations induced by partially directed edges (i.e., $\circ \longrightarrow$ and $\circ \circ$), the most plausible orientation is that which has the lowest entropy.

Our work extends the theoretic underpinnings of entropic causal discovery to generate a fully directed causal graph by resolving the partially directed edges produced by FCI. For each partially directed edge, we follow two steps: (1) establish if we can generate a latent variable (with low entropy) to serve as a common cause between two vertices; (2) if such a latent variable does not exist, then pick the causal direction which has the lowest entropy.

For the first step, we assess if there could be an unmeasured confounder (say Z) that lies between two partially oriented nodes (say X and Y). For this, we use the *LatentSearch* algorithm proposed by Kocaoglu *et al.* [64]. *LatentSearch* outputs a joint distribution $q(X, Y, Z)$ of the variables X , Y , and Z which can be used to compute the entropy $H(Z)$ of the unmeasured confounder Z . Following the guidelines of Kocaoglu *et al.*, we set an entropy threshold $\theta_r = 0.8 \times \min \{H(X), H(Y)\}$. If the entropy $H(Z)$ of the unmeasured confounder falls *below* this threshold, then we declare that there is a simple unmeasured confounder Z (with a low enough entropy) to serve as a common cause between X and Y and accordingly, we replace the partial edge with a bidirected (i.e., $\longleftrightarrow$) edge.

When there is no latent variable with a sufficiently low entropy, two possibilities exist: (a) variable X causes Y ; then, there is an arbitrary function $f(\cdot)$ such that $Y = f(X, E)$, where E is an exogenous variable (independent of X) that accounts for system noise; or (b) variable Y causes X ; then, there is an arbitrary function $g(\cdot)$ such that $X = g(Y, \tilde{E})$, where $\tilde{E}$ is an exogenous variable (independent of Y) that accounts for noise in the system. The distribution of E and $\tilde{E}$ can be inferred from the data [63, see §3.1]. With these distributions, we measure the entropies $H(E)$ and $H(\tilde{E})$. If $H(E) < H(\tilde{E})$, then, it is simpler to explain the $X \longrightarrow Y$ (i.e., the entropy is lower when $Y = f(X, E)$) and we choose $X \longrightarrow Y$. Otherwise, we choose $Y \longrightarrow X$.

Example. Fig. 10 shows the steps involved in generating the final ADMG. First, we build a complete undirected graph by connecting all pairs of variables with an undirected edge, where only a small subset of connections are shown for readability). Next, we use Fisher's exact test [22] to evaluate the independence of all pairs of variables conditioned on all remaining variables. Pruning edges between the independent variables results in a skeleton graph. Next, we orient undirected edges using edge orientation rules [21, 34, 78, 96] to produce a partial ancestral graph. In our example, we identify that there are two edges that are partially oriented: (i) Cache Misses $\circ \circ$ No of Cycles; and (ii) Cache Misses $\circ \longrightarrow$ Energy. To resolve these two edges, we use the entropic orientation strategy to orient these edges to get the final ADMG.

Stage-III: Iterative Sampling.

We extract paths from the causal graph (referred to as *causal paths*) and rank them from highest to lowest based on their average causal effect on latency, and energy. Using path extraction and ranking, we reduce the complex causal graph into a few useful causal paths for further analyses. The configurations in this path are more likely to be associated with the root cause of the fault.

Extracting causal paths with backtracking. A causal path is a directed path originating from either the configuration options or the system event and terminating at a non-functional property (i.e., throughput and/or energy). To discover causal paths, we backtrack from the nodes corresponding to each non-functional property until we reach a node with no parents. If any intermediate node has more than one parent, then we create a path for each parent and continue backtracking on each parent.

Ranking causal paths. A complex causal graph can result in many causal paths. It is not practical to reason over all possible paths, as it may lead to a combinatorial explosion. Therefore, we rank the paths in descending of their causal effect on each non-functional property. For further analysis, we use paths with the highest causal effect. To rank the paths, we measure the causal effect of changing the value of one node (say Batch Size or X) on its successor (say Cache Misses

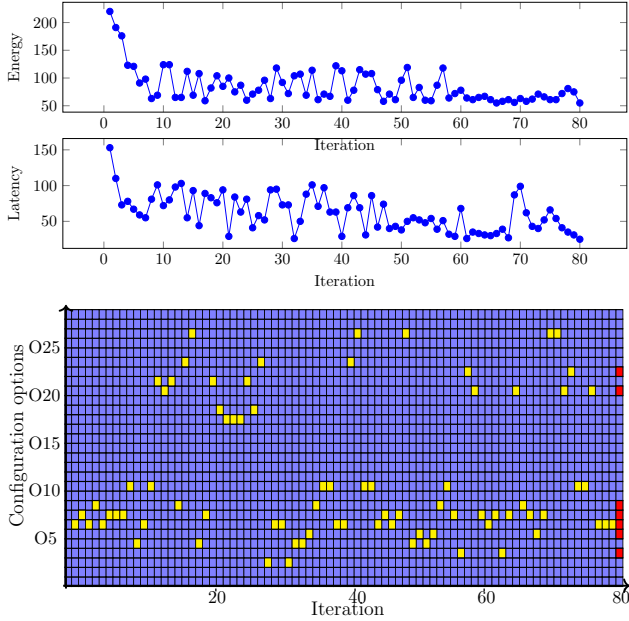


Figure 23. Incremental update of Latency and Energy using UNICORN for debugging a multi-objective fault (top two plots). Yellow-colored nodes indicate the configuration options, which their assigned value was changed based on the recommendation made by UNICORN at each particular iteration (bottom plot). Red colored nodes indicate the options that has been assigned a different values comparing with the corresponding value in the faulty configuration (Iteration 1.)

or Z) in the path (say Batch Size $\rightarrow$ Cache Misses $\rightarrow$ FPS and Energy). We express this with the *do-calculus* [81] notation: $\mathbb{E}[Z \mid do(X = x)]$. This notation represents the expected value of Z (Cache Misses) if we set the value of the node X (Batch Size) to x . To compute the *average causal effect* (ACE) of $X \rightarrow Z$ (i.e., Batch Size $\rightarrow$ Cache Misses), we find the average effect over all permissible values of X (Batch Size), i.e.,

$$ACE(Z, X) = \frac{1}{N} \cdot \sum_{a, b \in X} \mathbb{E}[Z \mid do(X = b)] - \mathbb{E}[Z \mid do(X = a)] \quad (1)$$

Here, N represents the total number of values X (Batch Size) can take. If changes in Batch Size result in a large change in Cache Misses, then $ACE(Z, X)$ will be larger, indicating that Batch Size on average has a large causal effect on Cache Misses. Note, if X is a continuous variable, we would replace the summation of Eq. (1) with an integral. For the entire path, we extend Eq. (1) as:

$$Path_{ACE} = \frac{1}{K} \cdot \sum ACE(Z, X) \quad \forall X, Z \in path \quad (2)$$

Eq. (2) represents the average causal effect of the causal path. The configuration options that lie in paths with larger P_{ACE} tend to have a greater causal effect on the corresponding non-functional properties in those paths. We select the top K

paths with the largest P_{ACE} values, for each non-functional property. In this paper, we use $K=3, 5, 7$ and 9 , however, this may be modified in our replication package.

Counterfactual queries can be different for different tasks. For debugging, we use the top K paths to (a) identify the root cause of non-functional faults; and (b) prescribe ways to fix the non-functional faults. Similarly, we use the top K paths to identify the options that can improve the non-functional property values near optimal. For both tasks, a developer may ask specific queries to UNICORN and expect an actionable response. For debugging, we use the example causal graph of where a developer observes low FPS and high energy, i.e., a multi-objective fault, and has the following questions:

❶ **“What are the root causes of my multi-objective (FPS and Energy) fault?”** To identify the root cause of a non-functional fault, we must identify which configuration options have the most causal effect on the performance objective. For this, we use the steps outlined in §3 to extract the paths from the causal graph and rank the paths based on their average causal effect (i.e., $Path_{ACE}$ from Eq. (2)) on latency and energy. We return the configurations that lie on the top K paths. For example, in Fig. 6 we may return (say) the following paths:

- Batch Size $\rightarrow$ Cache Misses $\rightarrow$ FPS and Energy
- Enable Padding $\rightarrow$ Cache Misses $\rightarrow$ FPS and Energy

and the configuration options BatchSize, and Enable Padding being the probable root causes.

❷ **“How to improve my FPS and Energy?”** To answer this query, we first find the root causes as described above. Next, we discover what values each of the configuration options must take in order that the new FPS and Energy is better (high FPS and low Energy) than the fault (low FPS and high Energy). For example, we consider the causal path Batch Size $\rightarrow$ Cache Misses $\rightarrow$ FPS and Energy, we identify the permitted values for the configuration options Batch Size that can result in a high FPS and energy (Y^{LOW}) that is better than the fault (Y^{HIGH}). For this, we formulate the following counterfactual expression:

$$\Pr(Y_{repair}^{LOW} \mid \neg repair, Y_{\neg repair}^{HIGH}) \quad (3)$$

Eq. (3) measures the probability of “fixing” the latency fault with a “repair” (Y_{repair}^{LOW}) given that with no repair we observed the fault ($Y_{\neg repair}^{HIGH}$). In our example, the repairs would resemble Batch Size=10. We generate a *repair set* ($\mathcal{R}_1$), where the configurations Batch Size is set to all permissible values, i.e.,

$$\mathcal{R}_1 \equiv \bigcup \{Batch\ Size = x, \dots\} \forall x \in Batch\ Size \quad (4)$$

observe that, in the repair set ($\mathcal{R}_1$) a configuration option that is not on the path Batch Size $\rightarrow$ Cache Misses $\rightarrow$ FPS and Energy is set to the same value of the fault. For example, Bit Rate is set to 2 or Enable Padding is set to 1. This way we can reason about the effect of interactions between Batch Size with other options, i.e., Bit Rate,

Buffer Size. Say Buffer Size or Enable padding were changed/recommended to set at any other value than the fault in some previous iteration, i.e., 20 or 0, respectively. In that case, we set BufferSize and Enable padding=0. Similarly, we generate a repair set $\mathcal{R}_2$ by setting Enable Padding to all permissible values.

$$\mathcal{R}_2 \equiv \bigcup \{ \text{Enable padding} = x, \dots \mid \forall x \in \text{Enable padding} \} \quad (5)$$

Now, we combine the repair set for each path to construct a final repair set $\mathcal{R} = \mathcal{R}_1 \cup \dots \cup \mathcal{R}_k$. Next, we compute the *Individual Causal Effect* (ICE) on the FPS and Energy (Y) for each repair in the repair set $\mathcal{R}$. In our case, for each repair $r \in \mathcal{R}$, ICE is given by:

$$\text{ICE}(r) = \Pr(Y_r^{\text{LOW}} \mid \neg r, Y_{\neg r}^{\text{HIGH}}) - \Pr(Y_r^{\text{HIGH}} \mid \neg r, Y_{\neg r}^{\text{HIGH}}) \quad (6)$$

ICE measures the difference between the probability that FPS and Energy is *low* after a repair r and the probability that the FPS and Energy is *still high* after a repair r . If this difference is positive, then the repair has a higher chance of fixing the fault. In contrast, if the difference is negative, then that repair will likely worsen both FPS and Energy. To find the most useful repair ($\mathcal{R}_{\text{best}}$), we find a repair with the largest (positive) ICE, i.e., $\mathcal{R}_{\text{best}} = \text{argmax}_{r \in \mathcal{R}} [\text{ICE}(r)]$. This provides the developer with a possible repair for the configuration options that can fix the multi-objective FPS and Energy fault.

Remarks. The ICE computation of Eq. (6) occurs *only* on the observational data. Therefore, we may generate any number of repairs and reason about them without having to deploy those interventions and measuring their performance in the real world. This offers significant runtime benefits.

11.3 Experimental setup (Additional details)

Table 4, Table 5, Table 6, and Table 7, show different software configuration options and their values for different systems considered in this paper. Table 8 shows the OS/kernel level configuration options and their values for different systems considered in this paper. Additionally, Table 10 shows the performance events considered in this paper. The hyperparameters considered for XCEPTION, BERT, and DEEPSPEECH are shown in Table 11.

We used the following four components for Deepstream implementation:

- **Decoder:** For the decoder, we use x264. It uses the x264 and takes the encoded H.64, VP8, VP9 streams and produces a NV12 stream.
- **Stream Mux:** The streammux module takes the NV12 stream and outputs the NV12 batched buffer with information about input frames, including original timestamp and frame number.
- **Nvinfer:** For object detection and classification, we use the TrafficCamNet model that uses ResNet 18 architecture. This model is pre-trained in 4 classes on a dataset of 150k

Table 4. Deepstream software configuration options.

Configuration Options	Values/Range
CRF	13, 18, 24, 30
Bit Rate	1000, 2000, 2800, 5000
Buffer Size	6000, 8000, 20000
Presets	ultrafast, veryfast, faster medium, slower
Maximum Rate	600k, 1000k
Refresh	OFF, ON
Batch Size	0-30
Batched Push Timeout	0-20
Num Surfaces per Frame	1,2,3,4
Enable Padding	0,1
Buffer Pool Size	1-26
Sync Inputs	0,1
Nvbuf Memory Type	0,1,2,3
Net Scale Factor	0.01 -10
Batch Size	1-60
Interval	1-20
Offset	0,1
Process Mode	0,1
Use DLA Core	0,1
Enable DLA	0,1
Enable DBSCAN	0,1
Secondary Reinfer Interval	0-20
Maintain Aspect Ratio	0,1
IOU Threshold	0-60
Enable Batch Process	0,1
Enable Past Frame	0,1
Compute HW	0,1,2,3,4

Table 5. Configuration options in XCEPTION, BERT, and DEEPSPEECH.

Configuration Options	Range
Memory Growth	0-1
Logical Devices	0,1

frames and has an accuracy of 83.5% for detecting and tracking cars from a traffic camera’s viewpoint. The 4 classes are Vehicle, BiCycle, Person, and Roadsign. We use the Keras (Tensorflow backend) pre-trained model from TensorRT.

- **Nvtracker:** The plugin accepts NV12- or RGBA-formatted frame data from the upstream component and scales (converts) the input buffer to a buffer in the format required by the low-level library, with tracker width and height. NvDCF tracker uses a correlation filter-based online discriminative learning algorithm as a visual object tracker, while using a data association algorithm for multi-object tracking.

Table 6. x264 software configuration options.

Configuration Options	Values/Range
CRF	13, 18, 24, 30
Bit Rate	1000, 2000, 2800, 5000
Buffer Size	6000, 8000, 20000
Presets	ultrafast, veryfast, faster medium, slower
Maximum Rate	600k, 1000k
Refresh	OFF, ON

Table 7. SQLite software configuration options.

Configuration Options	Range
PRAGMA TEMP_STORE	DEFAULT, FILE, MEMORY
PRAGMA JOURNAL_MODE	DELETE, TRUNCATE, PERSIST, MEMORY, OFF
PRAGMA SYNCHRONOUS	FULL, NORMAL, OFF
PRAGMA LOCKING_MODE	NORMAL, EXCLUSIVE
PRAGMA CACHE_SIZE	0, 1000, 2000, 4000, 10000
PRAGMA PAGE_SIZE	2048, 4096, 8192
PRAGMA MAX_PAGE_COUNT	32, 64
PRAGMA MMAP_SIZE	30000000000, 60000000000,

Table 8. Linux OS/Kernel configuration options.

Configuration Options	Range
vm.vfs_cache_pressure	1, 100, 500
vm.swappiness	10, 60, 90
vm.dirty_bytes	30, 60
vm.dirty_background_ratio	10, 80
vm.dirty_background_bytes	30, 60
vm.dirty_ratio	5, 50
vm.nr_hugepages	0, 1, 2
vm.overcommit_ratio	50, 80
vm.overcommit_memory	0, 2
vm.overcommit_hugepages	0, 1, 2
kernel.cpu_time_max_percent	10-100
kernel.max_pids	32768, 65536
kernel.numa_balancing	0, 1
kernel.sched_latency_ns	24000000, 48000000
kernel.sched_nr_migrate	32, 64, 128
kernel.sched_rt_period_us	1000000, 2000000
kernel.sched_rt_runtime_us	500000, 950000
kernel.sched_time_avg_ms	1000, 2000
kernel.sched_child_runs_first	0, 1
Swap Memory	1, 2, 3, 4 (GB)
Scheduler Policy	CFP, NOOP
Drop Caches	0, 1, 2, 3

Table 5 shows different software configuration options and their values for all components considered in this paper.

Table 6 shows different software configuration options and their values for each component considered in this paper.

Table 9. Hardware configuration options.

Configuration Options	Range	Description
CPU Cores	1-4	
CPU Frequency	0.3 - 2.0 (GHz)	
GPU Frequency	0.1-1.3 (GHz)	
EMC Frequency	0.1-1.8 (Ghz)	

Table 10. Performance system events and tracepoints.

System Events
Context Switches
Major Faults
Minor Faults
Migrations
Scheduler Wait Time
Scheduler Sleep Time
Cycles
Instructions
Number of Syscall Enter
Number of Syscall Exit
L1 dcache Load Misses
L1 dcache Loads
L1 dcache Stores
Branch Loads
Branch Loads Misses
Branch Misses
Cache References
Cache Misses
Emulation Faults
Tracepoint Subsystems
Block
Scheduler
IRQ
ext4

11.4 Evaluation (Additional details)

11.4.1 Case Study Fig. 25 shows the causal graph to resolve the real-world latency fault.

11.4.2 Effectiveness Table 13(a) shows the effectiveness of UNICORN in resolving single objective faults due to heat in NVIDIA TX1. Here, UNICORN outperforms correlation-based methods in all cases. For example, in BERT on TX1, UNICORN achieves 9% more accuracy, 11% more precision, and 10% more recall compared to the next best method, BUG-Doc. We observed heat gains as high as 7% (2% more than BUGDoc) on x264. The results confirm that UNICORN *can recommend repairs for faults that significantly improve latency and energy usage*. Applying the changes to the configurations recommended by UNICORN increases the performance drastically.

Table 11. Hyperparameters for DNNs used in UNICORN.

Hyperparameters	Range	Architecture
Number of filters entry flow	32	XCEPTION
Filter size entry flow	(3×3)	
Number of filters, middle flow	64	
Filter size middle flow	(3×3)	
Number of filters exit flow	728	
Filter size exit flow	(3×3)	
Batch Size	32	
Number of epochs	100	
Dropout	0.3	BERT
Maximum batch size	16	
Maximum sequence length	13	
Learning rate	$1e^{-4}$	
Weight decay	0.3	
Dropout	0.3	DEEPSPEECH
Maximum batch size	16	
Maximum sequence length	32	
Learning rate	$1e^{-4}$	
Number of epochs	10	

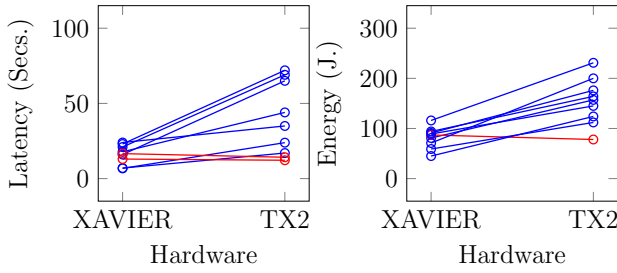


Figure 24. Ranking of configurations may change across environments, here between two hardware. The reason can be associated to differences in microarchitecture and different hardware resources. However, causal performance models capture the underlying causal mechanisms and therefore are able to capture the causal mechanisms and use them for performance related tasks in the new environments. On the other hand, performance influence models need to relearn the patterns from scratch, therefore, they demand for more sample in the new environments.

Table 12. Hyperparameters for FCI used in UNICORN.

Hyperparameters	Value
depth	-1
testId	fisher-z-test
maxPathLength	-1
completeRuleSetUsed	False

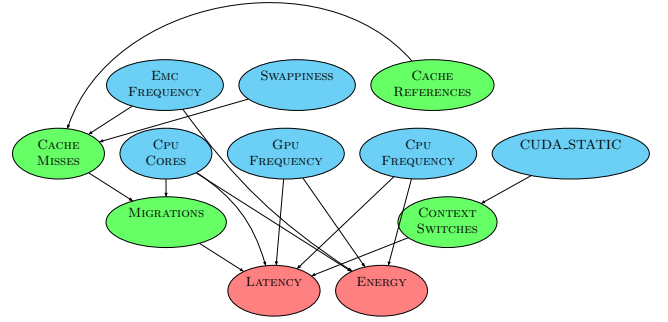


Figure 25. Causal graph used to resolve the latency fault in the real world case study.

UNICORN can resolve misconfiguration faults significantly faster than correlation-based approaches. In Table 13, the last two columns indicate the time taken (in hours) by each approach to diagnosing the root cause. UNICORN can do resolve faults significantly faster, e.g., UNICORN is 13× faster in diagnosing and resolving latency and heat faults for DEEPSPEECH.

11.4.3 Transferability Table. 15 indicates the results for different transfer scenarios: (I) We learn a causal model from TX1 and use them to resolve the latency faults in TX2, (I) We learn a causal model from TX2 and use them to resolve the energy faults in XAVIER, and (III) We learn a causal model from XAVIER and use them to resolve the heat faults in TX1. Here, we determine how transferable is UNICORN by comparing with UNICORN (Reuse), UNICORN +25, and UNICORN (Rerun). For all systems, we observe that performance of UNICORN (Reuse) is close to the performance of UNICORN (Rerun) which confirms the high transferability property of UNICORN. For example, in XCEPTION and SQLITE, UNICORN (Reuse) has the exact gain as of UNICORN (Rerun) for heat faults. For latency and energy faults, the gain difference between UNICORN (Reuse) and UNICORN (Rerun) is less than 5% for all systems. We also observe that with little updates, UNICORN +25 (~24 minutes) achieves a similar performance of UNICORN (RERUN) (~40 minutes), on average. This confirms that as the causal mechanisms are sparse, the CPM from source in UNICORN quickly reaches a fixed structure in the target using incremental learning by judiciously evaluating the most promising fixes until the fault is resolved.

11.4.4 Scalability Scalability of UNICORN depends on the scalability of each phase. Therefore, we design scenarios to test the scalability of each phase to determine the overall scalability. Since the initial number of samples and the underlying phases for each task is the same, it is sufficient to examine the scalability of UNICORN for the debugging non-functional fault task.

SQLITE was chosen because it offers a large number of configurable options, much more than neural applications,

Table 13. Efficiency of UNICORN compared to other approaches. Cells highlighted in **blue** indicate improvement over faults.

(a) Single objective performance fault in heat.

			Accuracy					Precision					Recall					Gain					Time [†]	
			UNICORN	CBI	DD	ENCORE	BugDoc	UNICORN	CBI	DD	ENCORE	BugDoc	UNICORN	CBI	DD	ENCORE	BugDoc	UNICORN	CBI	DD	ENCORE	BugDoc	UNICORN	Others
TX1	Heat	XCEPTION	69	63	57	64	65	75	56	56	60	66	68	62	58	64	69	4	3	2	2	3	0.6	4
		BERT	71	62	61	61	62	72	56	59	56	61	72	65	62	67	62	5	3	2	2	3	0.4	4
		DEEPSPEECH	71	61	64	62	67	71	58	59	54	68	69	67	66	68	67	3	3	2	2	2	0.7	4
		x264	74	65	57	64	65	74	62	54	55	65	74	66	63	68	69	7	3	2	2	5	1.4	4

(b) Multi-objective non-functional faults in Heat, Latency.

			Accuracy				Precision				Recall				Gain (Latency)				Gain (Heat)				Time [†]	
			UNICORN	CBI	ENCORE	BugDoc	UNICORN	CBI	ENCORE	BugDoc	UNICORN	CBI	ENCORE	BugDoc	UNICORN	CBI	ENCORE	BugDoc	UNICORN	CBI	ENCORE	BugDoc	UNICORN	Others
Latency + Heat		XCEPTION	62	52	55	57	69	57	50	61	61	48	51	60	58	42	47	51	2	1	1	1	0.9	4
		BERT	64	52	47	56	62	52	45	60	68	54	62	65	65	37	48	60	4	3	2	3	0.4	4
		DEEPSPEECH	62	52	43	55	60	48	48	55	67	58	41	59	69	37	45	65	4	1	1	4	0.3	4
		x264	61	53	53	60	63	50	54	61	60	53	55	55	67	54	54	65	5	3	3	4	0.5	4

(c) Multi-objective non-functional faults in Energy, Heat.

			Accuracy				Precision				Recall				Gain (Energy)				Gain (Heat)				Time [†]	
			UNICORN	CBI	ENCORE	BugDoc	UNICORN	CBI	ENCORE	BugDoc	UNICORN	CBI	ENCORE	BugDoc	UNICORN	CBI	ENCORE	BugDoc	UNICORN	CBI	ENCORE	BugDoc	UNICORN	Others
Energy + Heat		XCEPTION	65	55	57	63	64	55	51	62	67	47	53	60	58	44	51	54	3	1	1	1	0.8	4
		BERT	69	55	51	59	65	53	47	61	71	53	61	67	65	41	51	61	4	2	2	3	0.4	4
		DEEPSPEECH	72	55	49	61	73	51	51	61	71	57	53	64	69	47	51	64	4	1	1	3	0.3	4
		x264	72	59	57	66	71	51	55	62	69	61	59	59	67	51	51	61	5	2	3	4	0.5	4

Table 14. Efficiency of UNICORN in detecting and repairing the root-cause of multiple non-functional faults: and Energy, Latency, Heat. Cells highlighted in **green** indicate improvement over faults and **red** indicate deterioration. UNICORN achieves better performance overall and is much faster. Note: the results are reported for NVIDIA Jetson TX2.

			Accuracy				Precision				Recall				Gain (Latency)				Gain (Energy)				Gain (Heat)				Time [†]	
			UNICORN	CBI	ENCORE	BugDoc	UNICORN	CBI	ENCORE	BugDoc	UNICORN	CBI	ENCORE	BugDoc	UNICORN	CBI	ENCORE	BugDoc	UNICORN	CBI	ENCORE	BugDoc	UNICORN	CBI	ENCORE	BugDoc	UNICORN	Others
All Three	Image x264	Image	76	57	48	66	68	61	57	61	81	53	46	70	62	33	30	42	52	23	18	24	4	1	0	0	0.1	4
		x264	80	59	47	54	76	61	56	63	81	56	46	51	12	2	1	2	15	4	2	4	4	1	0	1	0.1	4
		SQLite	73	56	51	53	68	59	56	60	78	54	45	51	12	1	1	4	8	4	2	5	1	1	-1	-1	0.1	4

[†] Wallclock time in hours

and video encoders. Further, each of these options can take on a large number of permitted values, making DEEPSTREAM a useful candidate to study the scalability of UNICORN. DEEPSTREAM was chosen as it has a higher number of components than others, and it is interesting to determine how UNICORN it behaves when the number of options and events are increasing. As a result, SQLite exposes new system design

opportunities to enable efficient inference and many complex interactions between software options.

In large systems, there are significantly more causal paths and therefore, causal learning and estimations of queries take more time. However, with as much as 242 configuration options and 19 events (Table 3, row 2), causal graph discovery takes roughly one minute, evaluating all 2234 queries takes roughly two minutes, and the total time to diagnose and fix a

Table 15. (RQ3) Transferring causal models across hardware platforms.

TX1 (source) $\rightarrow$ TX2 (target)													
		Accuracy			Recall			Precision			Δ_{gain}		
		UNICORN (REUSE)	UNICORN +25	UNICORN (RERUN)	UNICORN (REUSE)	UNICORN +25	UNICORN (RERUN)	UNICORN (REUSE)	UNICORN +25	UNICORN (RERUN)	UNICORN (REUSE)	UNICORN +25	UNICORN (RERUN)
Software													
Latency	XCEPTION	52	83	86	70	79	86	46	78	83	46	71	82
	BERT	55	75	81	57	70	71	45	67	76	43	70	74
	DEEPSPEECH	45	71	81	56	79	81	49	73	76	54	73	76
	x264	57	79	83	70	75	78	58	77	82	45	73	85
TX2 (source) $\rightarrow$ XAVIER (target)													
Energy	XCEPTION	53	74	84	48	73	80	51	69	78	43	73	83
	BERT	50	61	66	53	71	79	49	66	70	40	55	62
	DEEPSPEECH	57	70	73	45	74	78	43	69	75	49	71	78
	x264	54	72	77	46	72	78	42	75	83	46	79	87
XAVIER (source) $\rightarrow$ TX1 (target)													
Heat	XCEPTION	63	64	69	61	67	68	58	74	75	3	4	4
	BERT	55	65	71	59	67	72	52	64	72	3	4	5
	DEEPSPEECH	57	64	71	59	63	69	53	63	71	1	2	3
	x264	51	65	74	53	64	74	54	62	74	3	5	7

fault is roughly 22 minutes for SQLite. This trend is observed even with 242 configuration options, 288 events (Table 3, row 3), and finer granularity of configuration values—the time required to causal model recovery is a little over 1 minute and the total time to diagnose and fix a fault is less than 2 hours. Similarly, in DEEPSTREAM, with 53 configuration options and 288 events, causal model discovery is less than two minutes and the time needed to diagnose and fix a fault is less than an hour. The results in Table 3 indicate that UNICORN can scale to a much larger configuration space without an exponential increase in runtime for any of the intermediate stages. This can be attributed to the sparsity of the causal graph (average degree of a node for SQLite in Table 3 is at most 3.6, and it reduces to 1.6 when the number of configurations increase and reduces from 3.1 to 2.3 in DEEPSTREAM when systems events are increased). This makes sense because not all variables (i.e., configuration options and/or system events) affect non-functional properties and a high number of variables in the graph end up as isolated nodes. Therefore, the number of paths and consequently the evaluation time do not grow exponentially as the number of variables increase.

Finally, the latency gain associated with repairs from larger configuration space with configurations was similar to the original space of 34 and 53 configurations for SQLite and DEEPSTREAM, respectively. This indicates that: (a) imparting domain expertise to select most important configuration options can speed up the inference time of UNICORN, and (b) if the user chooses instead to use more configuration options

(perhaps to avoid initial feature engineering), UNICORN can still diagnose and fix faults satisfactorily within a reasonable time.